FRAMESTUDYO

Project Report

A JavaFX Desktop Graphic Design Application

Design freely • Create Boldly

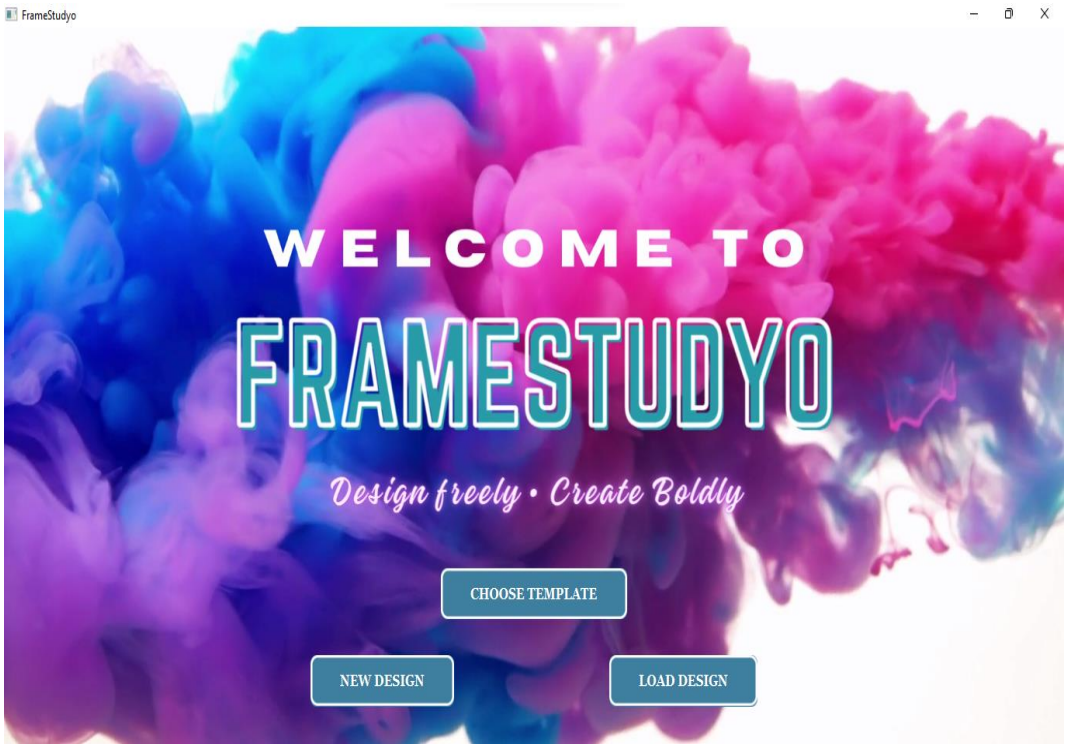

Figure 1 – FrameStudyo Welcome Screen

Prepared by: Ayesha Rahman

Date: 22 May 2026

Technology: Java 17 + JavaFX | Built in IntelliJ IDEA

Table of Contents

1. Project Overview

FrameStudyo is a feature-rich, JavaFX-based desktop graphic design application developed as an academic project. Inspired by tools such as Canva, it provides a fully interactive canvas on which users can compose original designs using shapes, text, images, and freehand drawing, supported by an intelligent suite of smart tools for colour guidance, alignment snapping, and automated design scoring.

The application was built using the Model-View-Controller pattern and demonstrates core software-engineering principles including object-oriented design, the Command pattern for undo/redo, polymorphism across all shape types, and singleton-based session analytics.

1.1 Objectives

- Provide an intuitive, drag-and-drop design canvas with a professional UI.
- Implement six distinct shape types with full property editing (colour, stroke, opacity, rotation).
- Deliver smart tools: colour palette advisor, auto snap-align, freehand shape recognition, design scorer, and a template engine.
- Track every user action in a live session analytics panel.
- Support file persistence (save/load .fsd), PNG export, and a template-based onboarding flow.

1.2 Technology Stack

Component	Detail
Language	Java 17
UI Framework	JavaFX 21
IDE	IntelliJ IDEA 2025
Build Tool	Maven
Serialisation	Java Object Serialisation (.fsd files)
Image Export	javax.imageio – PNG snapshot
Pattern	Command, Singleton, Polymorphism (OOP)

2. Application Architecture

FrameStudyo is organised into nineteen Java source files grouped by responsibility. The architecture follows a layered approach: a UI layer (FRAMESTUDYO, WelcomeScreen), a canvas/model layer (DesignCanvas, DesignElement and its subclasses), a command layer (Commands), a smart-tools layer (AlignmentEngine, ShapeRecognizer, SmartColorAdvisor, DesignAnalyzer, TemplateEngine), and an analytics/persistence layer (SessionTracker, FileManager).

2.1 Class Overview

Class / File	Category	Responsibility
FRAMESTUDYO.java	UI / Entry Point	Main Application class, toolbar, mouse events, layout
WelcomeScreen.java	UI	Splash screen with video background, template launcher
DesignCanvas.java	Model	Canvas wrapper; holds element list, redraw, z-ordering
DesignElement.java	Model (Abstract)	Base class for all shapes; position, colour, opacity, rotation
RectangleElement.java	Shape	Draws filled/stroked rectangles with rotation
CircleElement.java	Shape	Draws filled/stroked ovals/circles with rotation
LineElement.java	Shape	Draws line segments with click-tolerance hit test
TextElement.java	Shape	Renders text with custom font family and size
ImageElement.java	Shape	Embeds external images with lazy loading
Shapes.java	Shape	Contains TriangleElement, StarElement, ArrowElement
Commands.java	Command Pattern	Add, Delete, Move, Clear commands + CommandManager undo/redo stack
AlignmentEngine.java	Smart Tool	Snap-to-align drag logic with cyan guide overlays
ShapeRecognizer.java	Smart Tool	Freehand stroke classification into rectangle/circle/line/star/triangle
SmartColorAdvisor.java	Smart Tool	HSL-based complementary/triadic palette suggestions
DesignAnalyzer.java	Smart Tool	Scores balance, contrast, density, complexity (0–100)
TemplateEngine.java	Smart Tool	Five pre-built design templates as element lists
SessionTracker.java	Analytics	Singleton tracking session time, action log, shape stats
FileManager.java	Persistence	Java serialisation save/load of element lists (.fsd)

2.2 OOP Design Principles

The project makes deliberate use of the four pillars of object-oriented programming:

- Abstraction – DesignElement is abstract; it defines the contract (draw, duplicate, contains) without specifying how each shape renders.
- Polymorphism – DesignCanvas.redraw() calls element.draw(gc) on every element; each subclass handles rendering differently without any cast or type check.
- Encapsulation – Every field in DesignElement is protected with public getters/setters; the canvas internal state is hidden behind DesignCanvas methods.
- Inheritance – All six concrete shape types extend DesignElement and inherit position, colour, opacity, and rotation without duplication.

3. User Interface Walkthrough

3.1 Welcome Screen

When the application launches, a full-screen welcome screen is displayed with an animated video background. Three buttons are presented: NEW DESIGN, LOAD DESIGN, and CHOOSE TEMPLATE. The template chooser opens a secondary popup listing all available layout templates.

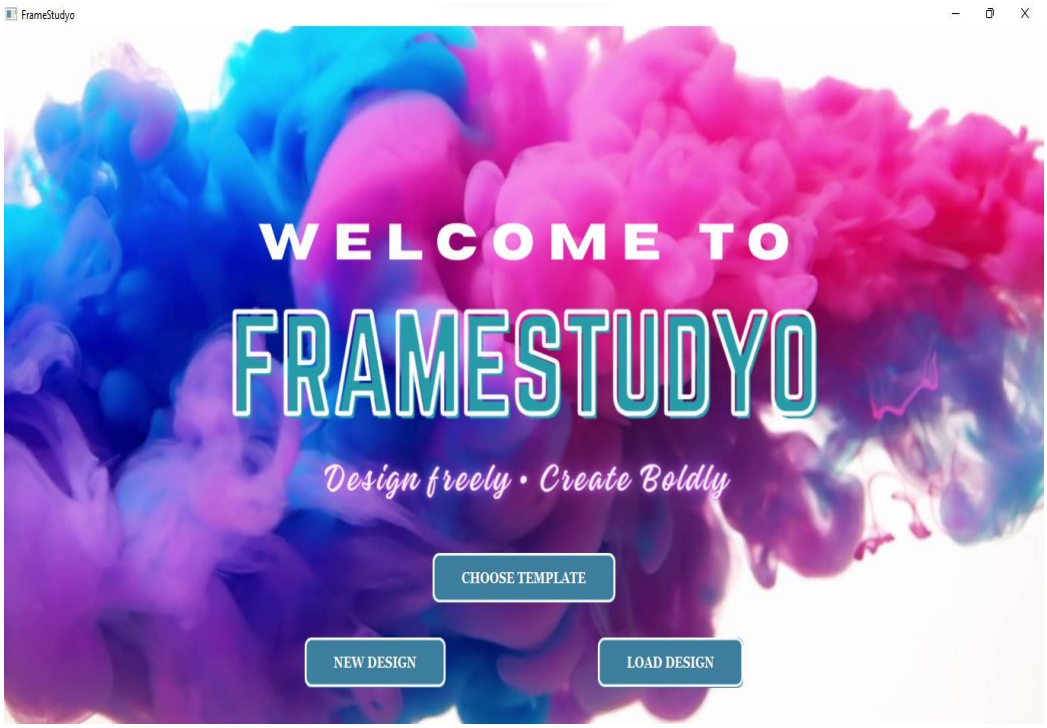

Figure 2 – Welcome Screen with animated background

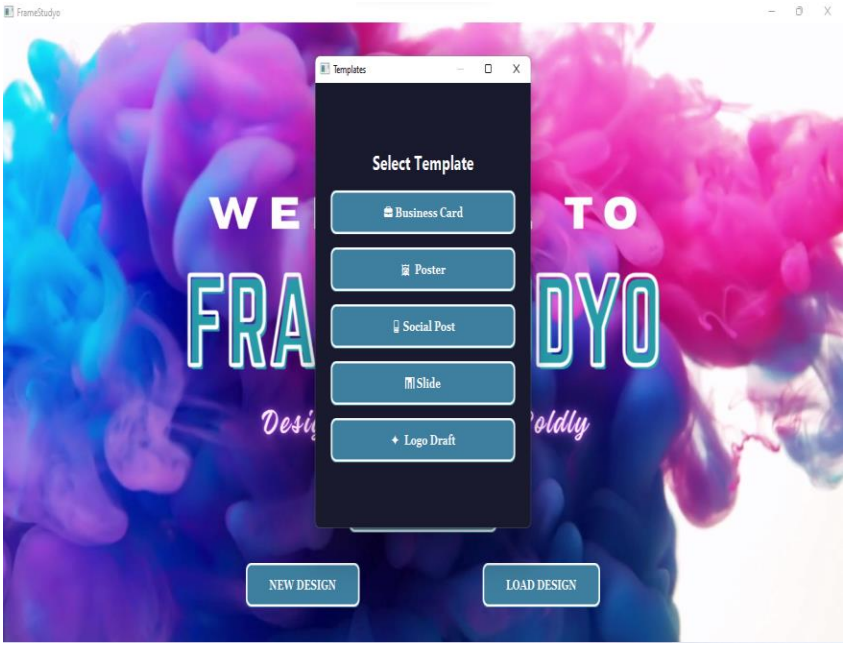

Figure 3 – Template selection dialog

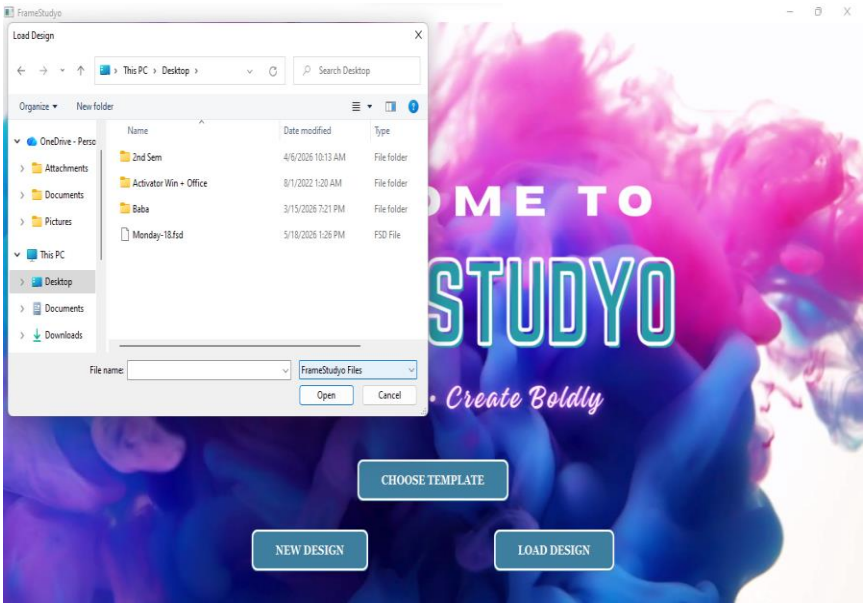

Figure 4 – Load Design file chooser (native OS dialog, .fsd filter)

3.2 Main Canvas & Toolbar

The main editor presents an 860x600 pixel white canvas centred on an animated dark background. The top toolbar provides all major tools in a logical left-to-right order: File operations, shape drawing, selection, text, freehand drawing, image import, element actions, undo/redo, smart tools, and canvas settings. The status bar at the bottom shows live mouse coordinates, the active tool, and the selected element type.

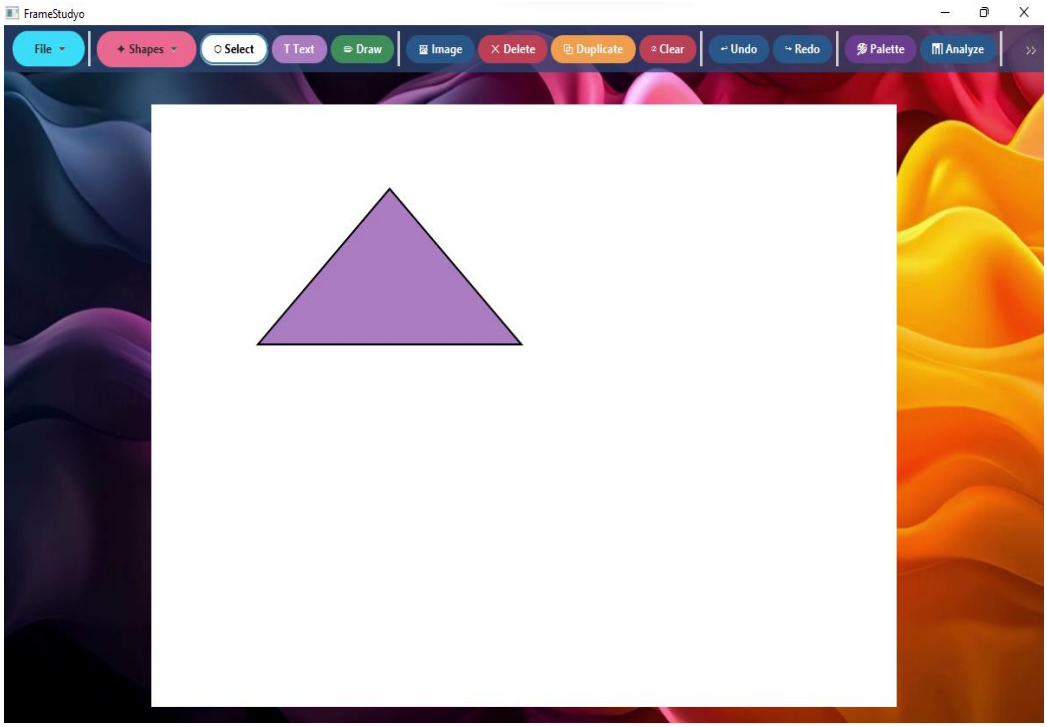

Figure 5 – Main canvas with a drawn triangle; clean toolbar layout

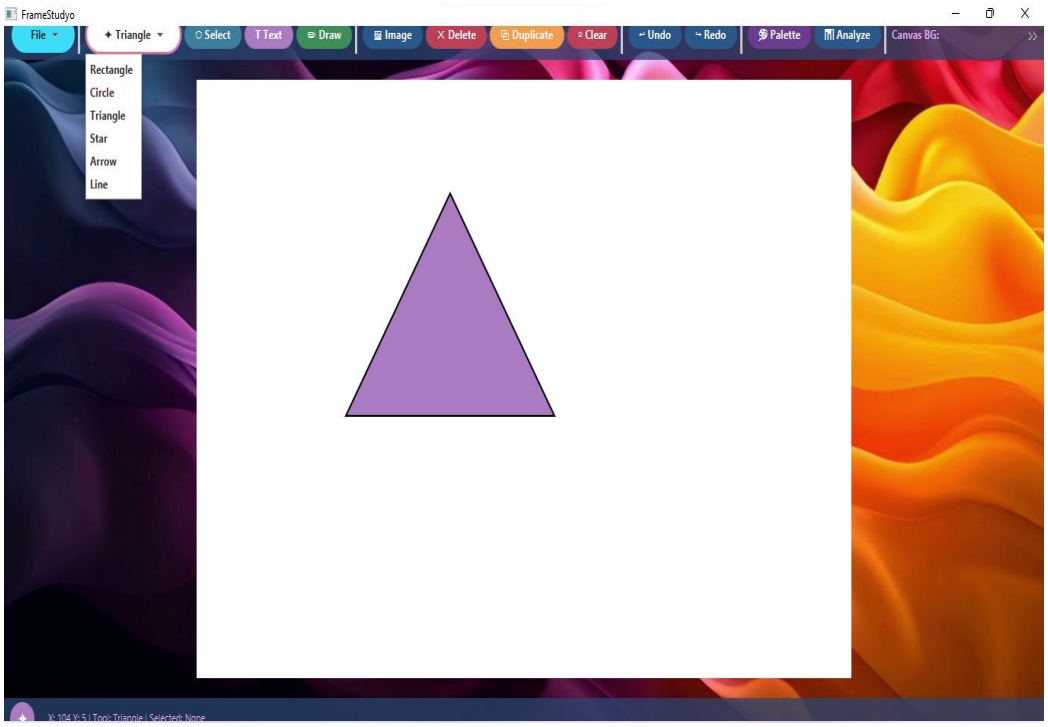

Figure 6 – Shapes dropdown menu expanded showing all six shape types

3.3 Selection & Properties Panel

Clicking an element with the Select tool highlights it with a cyan dashed bounding box and white corner handles that can be dragged to resize. The right-hand Properties panel activates, exposing fill colour, stroke colour, stroke width, opacity, and rotation sliders. Below those controls, the Canvas and Layers sections allow grid toggling and z-order management respectively.

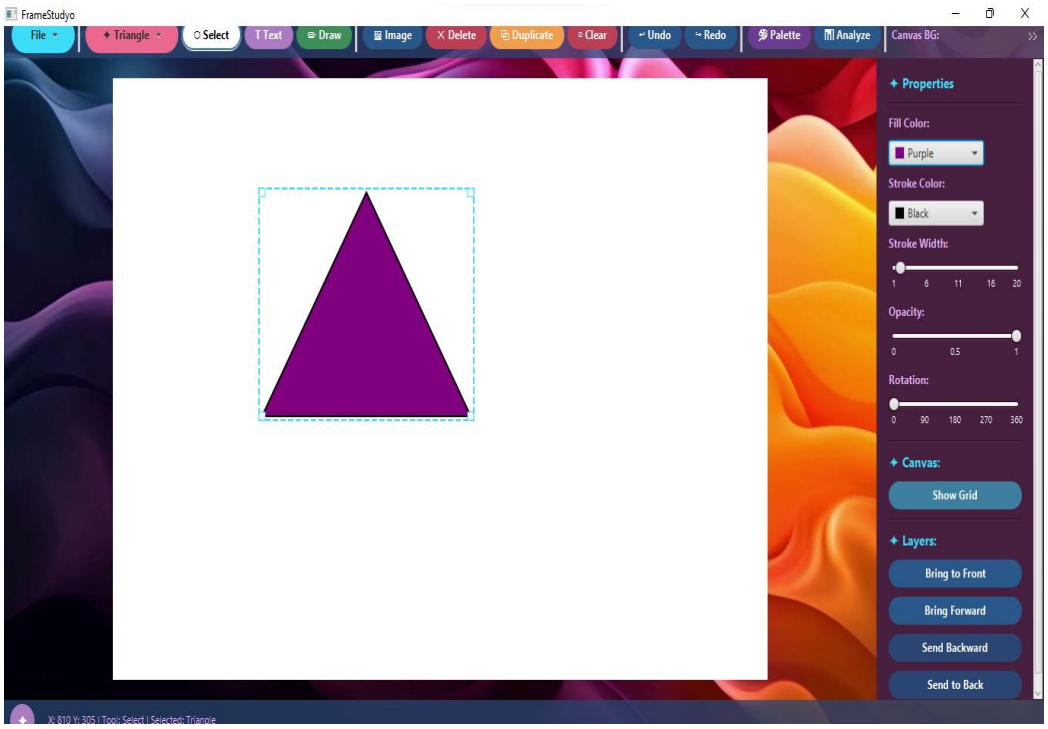

Figure 7 – Element selected with handles visible; Properties panel active

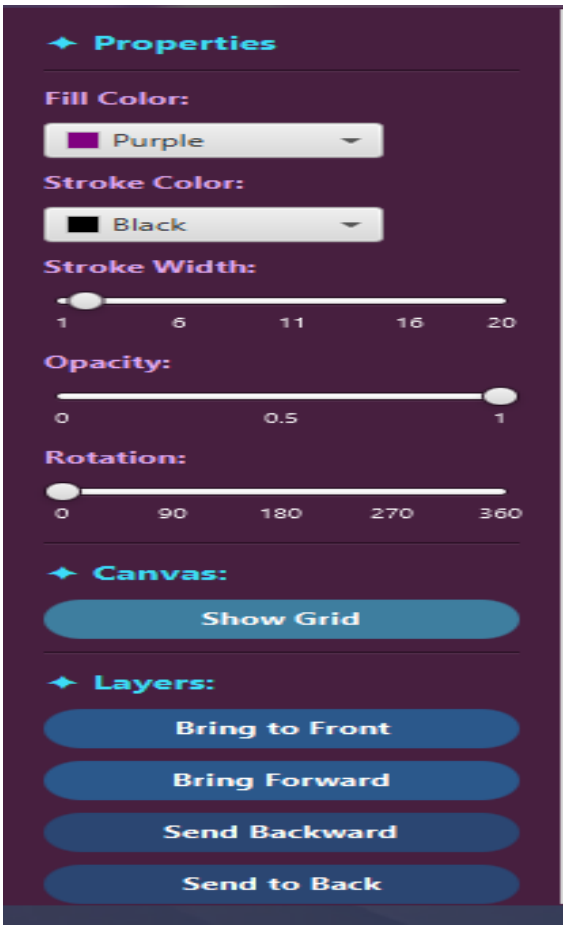

Figure 8 – Properties panel close-up showing all sliders and layer controls

3.4 Text & Multi-Element Canvas

The text tool opens a dialog asking for the text content, font family (eight choices), and font size. Once placed, text elements are fully movable, selectable, and property-editable just like any shape.

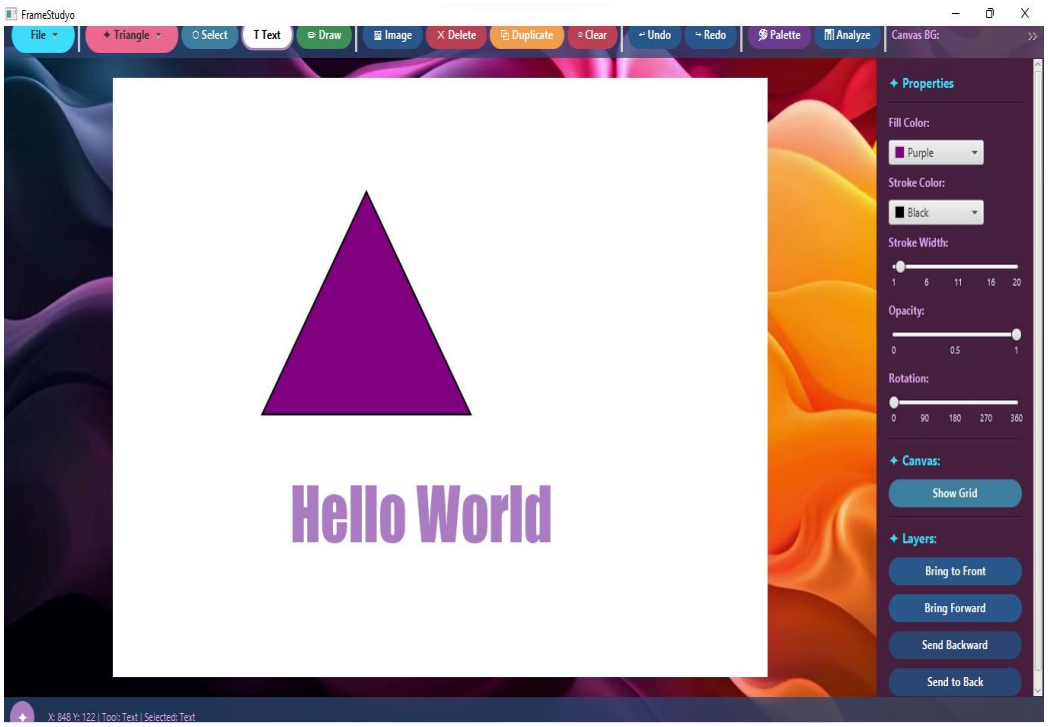

Figure 9 – Canvas with a triangle, text element "Hello World", and the Text tool active

3.5 Image Import

The Image button in the toolbar opens a native file picker accepting PNG, JPG, JPEG, and GIF files. The image is embedded as an ImageElement on the canvas at position (100, 100) and can be resized, moved, or rotated just like any other element.

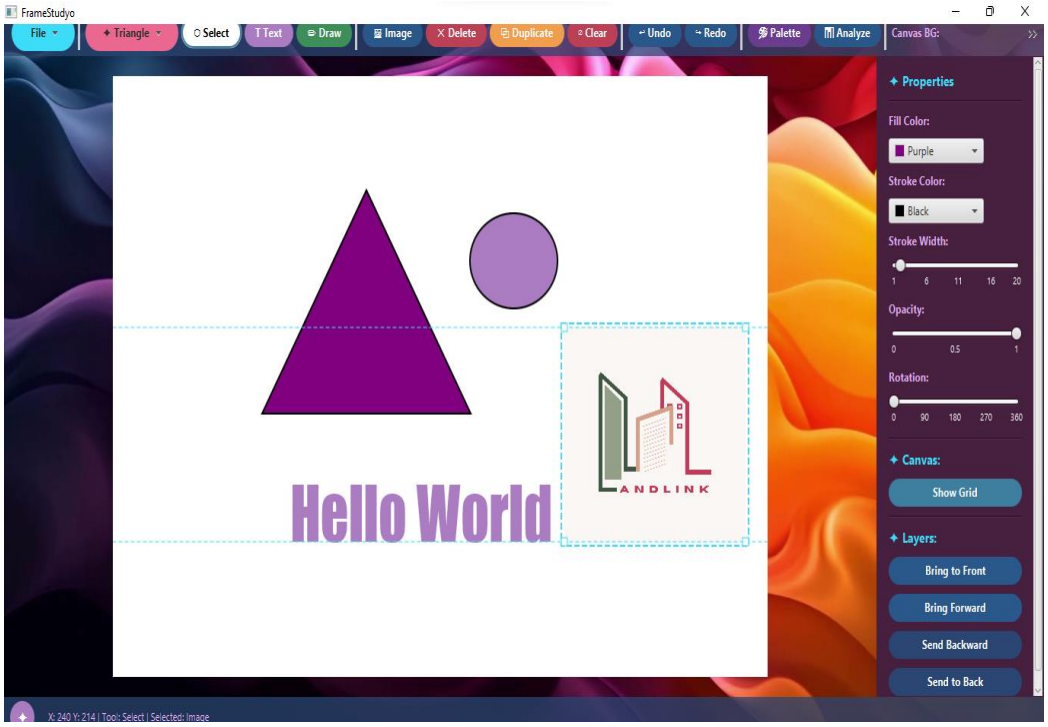

Figure 10 – Canvas after importing an external PNG logo image

3.6 Canvas Background & Grid

A colour picker in the toolbar ("Canvas BG:") lets users set any background colour. The grid toggle overlays a 20-pixel equidistant grid to aid precise placement. Both settings are reflected immediately in the canvas redraw.

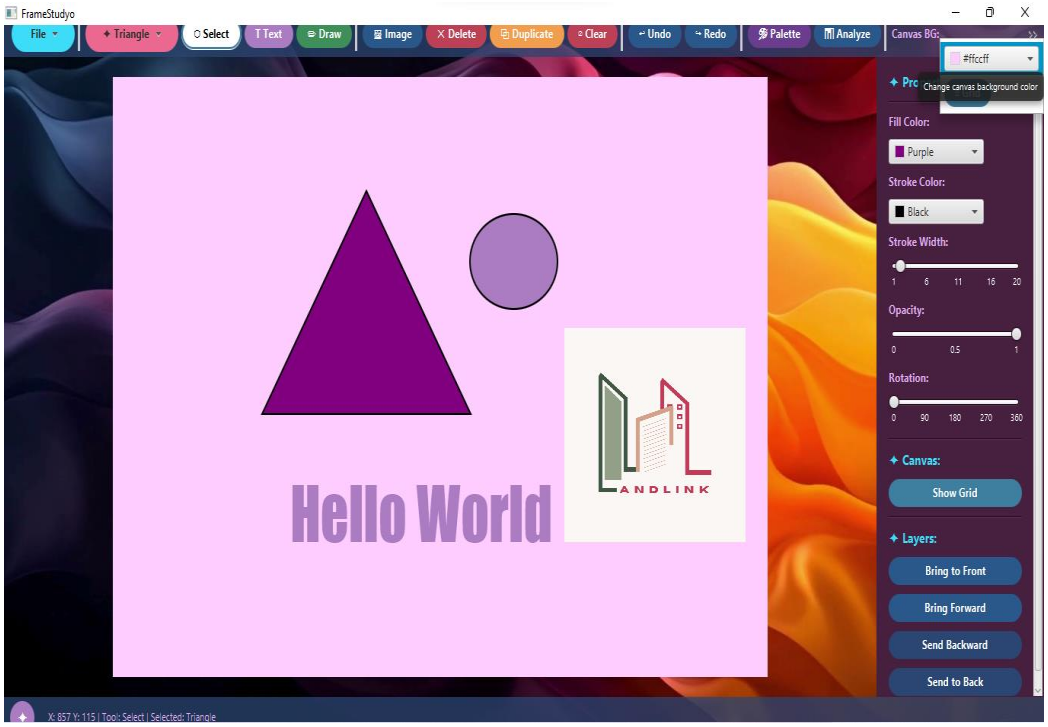

Figure 11 – Canvas background changed to a custom pink colour

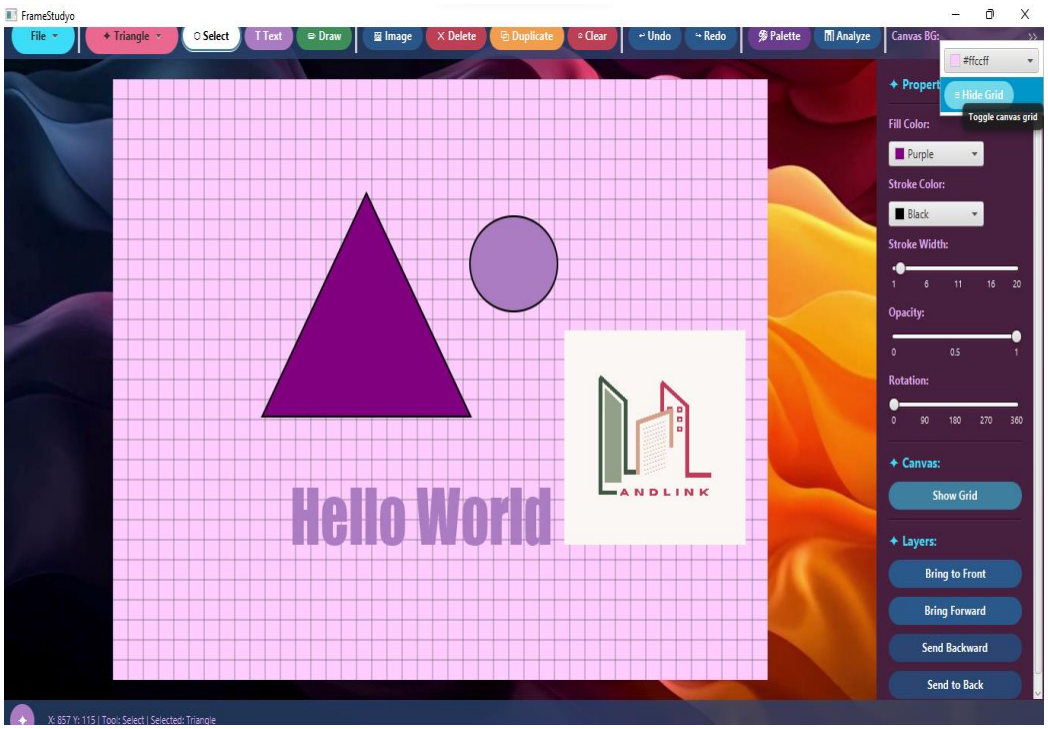

Figure 12 – Grid overlay enabled on the canvas

4. Smart Tools

4.1 Freehand Shape Recognizer

The Draw tool records a freehand stroke as a sequence of (x, y) points sampled every 3 pixels. On mouse release, ShapeRecognizer classifies the stroke by computing its bounding box, aspect ratio, linearity deviation, closure test, approximate corner count, and a circularity metric. The resulting element (rectangle, circle, triangle, star, or line) is added to the canvas via the Command pattern and the status bar displays the recognised label.

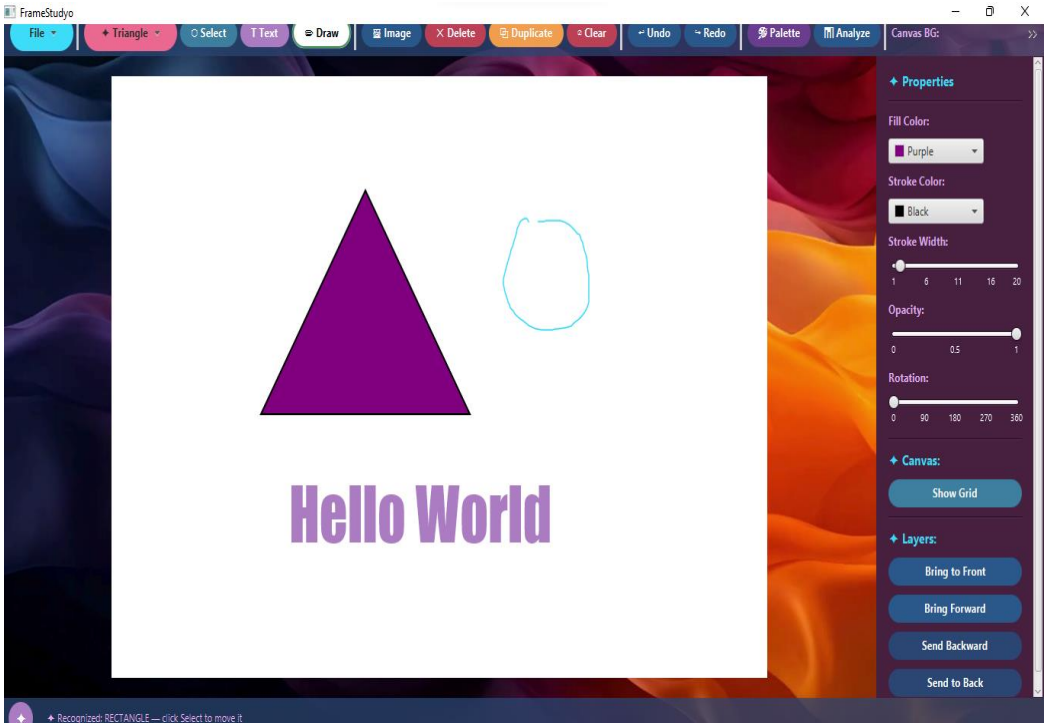

Figure 13 – Freehand drawing in progress; status bar shows "Recognized: RECTANGLE"

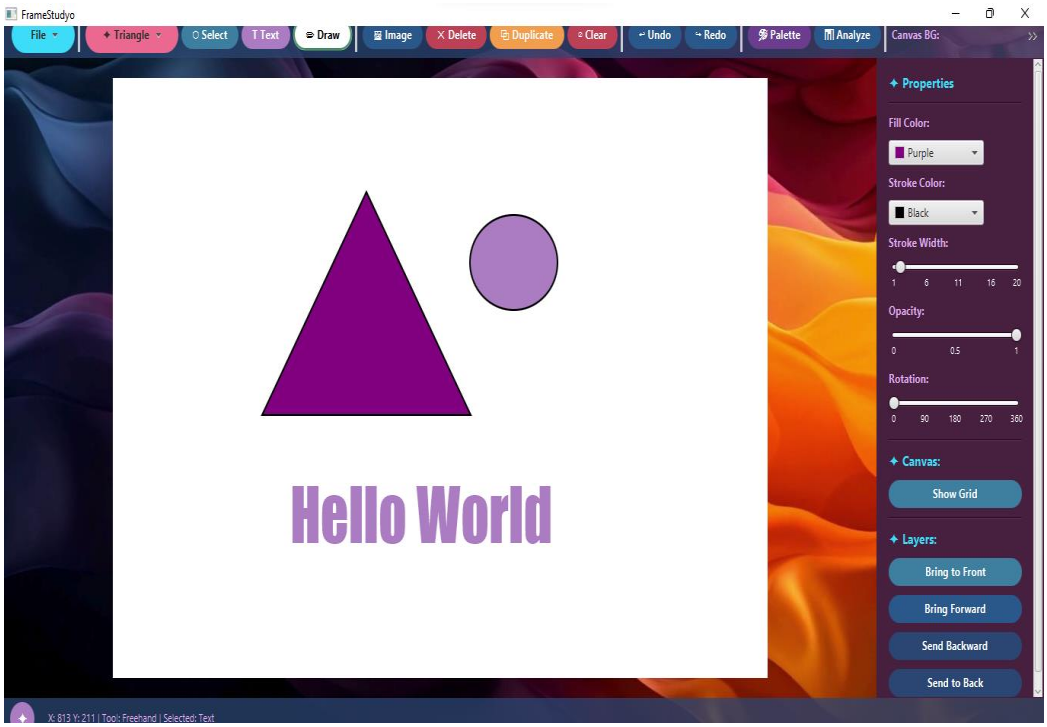

Figure 14 – Recognised circle placed on canvas after freehand stroke

4.2 Smart Color Palette Advisor

The Palette button triggers SmartColorAdvisor, which extracts the unique fill colours currently on the canvas and generates up to six complementary suggestions using HSL colour theory: a 180° complementary, two triadic splits (+120° and +240°), and a lighter tint. Colour swatches are displayed in a floating panel; clicking any swatch copies its hex code to the clipboard.

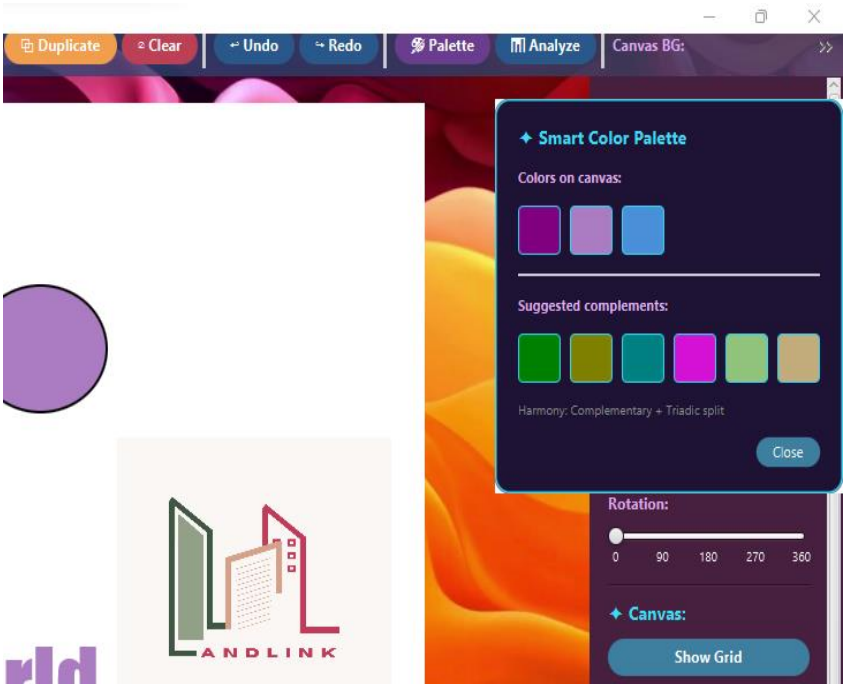

Figure 15 – Smart Color Palette showing canvas colours and complementary suggestions

4.3 Design Analyzer

The Analyze button opens DesignAnalyzer, which scores the current design on four axes, each from 0 to 100, averaged into an overall score. The scores are displayed with colour-coded progress bars (green above 75%, yellow above 50%, red below).

- Balance – Distance of the visual centre-of-mass from the canvas centre.
- Contrast – Colour luminance variance across all filled elements.
- Density – Percentage of canvas area covered by elements (ideal range: 20–60%).
- Complexity – Element count (ideal range: 3–12 elements).

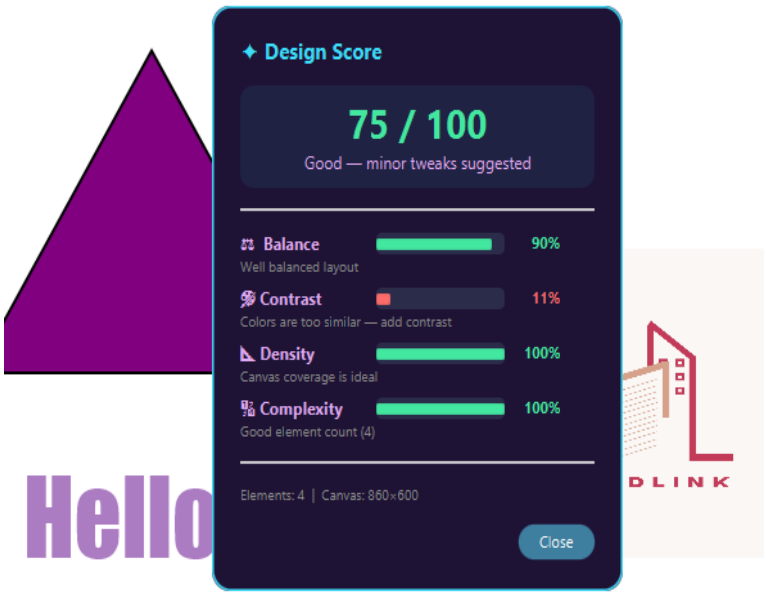

Figure 16 – Design Analyzer showing a score of 75/100 with per-axis feedback

4.4 Alignment Engine (Smart Snap)

While dragging an element with the Select tool, AlignmentEngine compares the dragged element's left, right, centre-X, top, bottom, and centre-Y edges against all other elements. When any pair falls within 8 pixels, the element snaps to the exact alignment position and cyan dashed guide lines are drawn across the full canvas width or height. On mouse release, guides are cleared.

4.5 Template Engine

TemplateEngine provides five built-in layouts, each constructed entirely from DesignElement subclasses without any image assets: Business Card, Poster, Social Post, Presentation Slide, and Logo Draft. Choosing a template from the welcome screen or the template popup replaces the canvas contents with the template elements, which can then be freely edited.

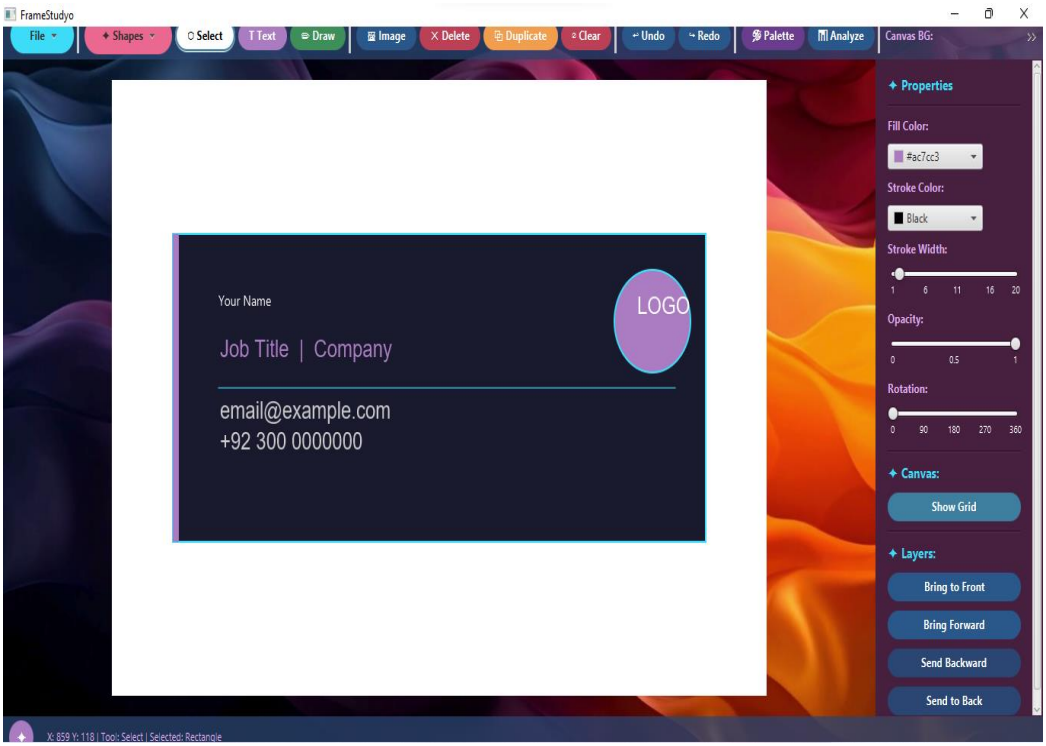

Figure 17 – Business Card template loaded into the canvas

5. Session Tracker & Analytics

SessionTracker is implemented as a thread-safe singleton. It is started when the application fully loads and records the wall-clock start time. Every meaningful user action (adding a shape, deleting, duplicating, saving, exporting, drawing, etc.) is appended to an in-memory action log as an ActionEntry with a timestamp.

The Stats panel (the >> button in the toolbar) opens a floating overlay showing four metric tiles – session time (updated every second via a Timeline animation), element count, total actions, and canvas coverage percentage – followed by a shape-type breakdown, the most-used fill colour, and a scrollable recent-actions history log.

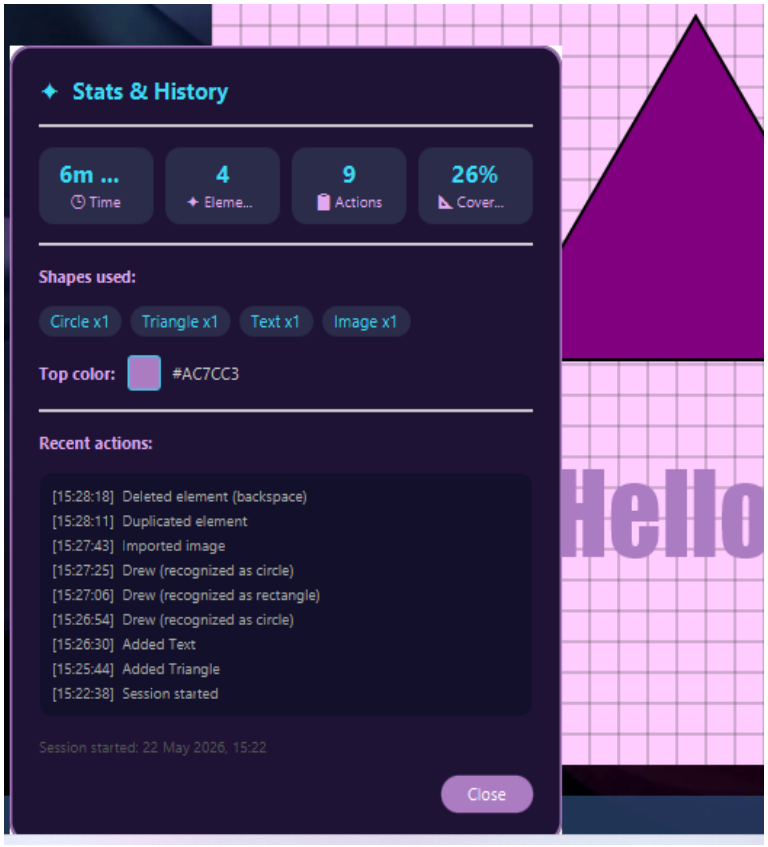

Figure 18 – Stats & History panel showing live session data

5.1 Computed Statistics

- Session Time – Elapsed wall-clock time since startSession() was called.
- Element Count – Number of DesignElement objects currently on the canvas.
- Action Count – Length of the history list.
- Canvas Coverage – Sum of all element bounding areas / (860 x 600), capped at 100%.
- Shape Breakdown – Grouped count by class name (e.g., Circle x1, Triangle x1).
- Most-Used Colour – Fill hex string appearing in the highest number of elements.

6. File Management & Export

All persistence is handled by FileManager using Java's built-in ObjectOutputStream / ObjectInputStream. The entire elements list is serialised to a .fsd (FrameStudyo Design) file. DesignElement implements Serializable; ImageElement marks its transient Image field with the transient keyword and re-loads it lazily from the stored file path on next draw.

An InvalidClassException handler detects files saved with older class versions and displays an informative error dialog rather than crashing. PNG export uses WritableImage and javax.imageio to snapshot the canvas pixel-by-pixel, preserving transparency.

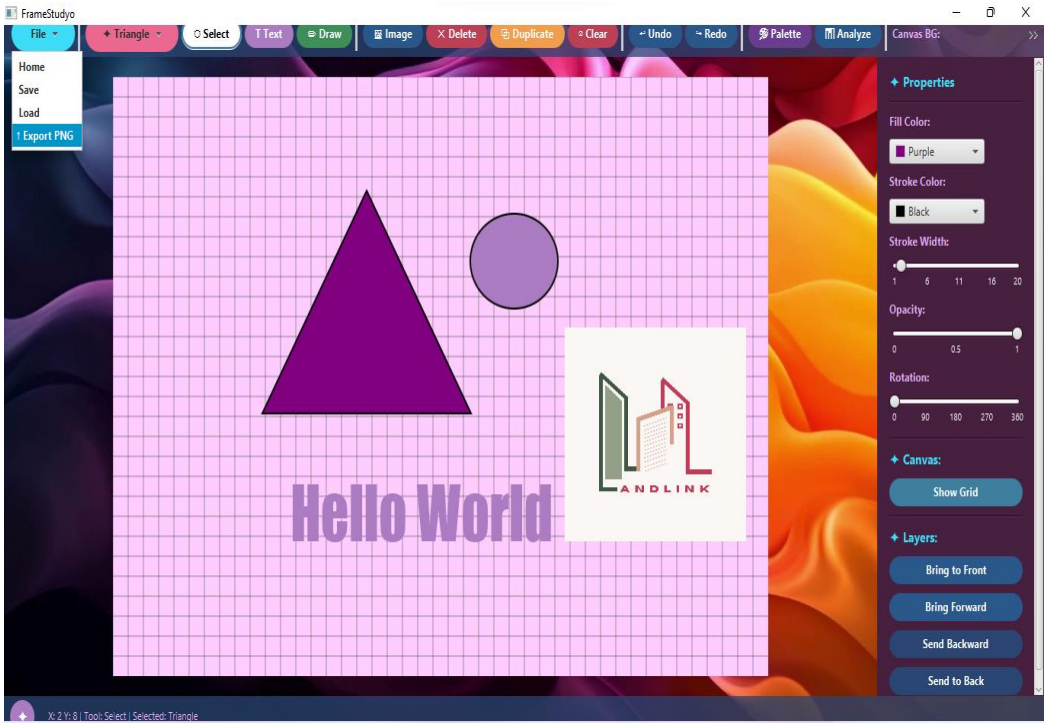

Figure 19 – File menu showing Save, Load, Home, and Export PNG options

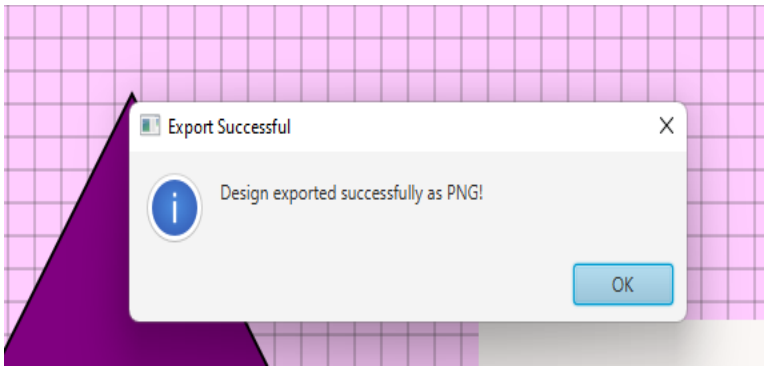

Figure 20 – Export Successful confirmation dialog

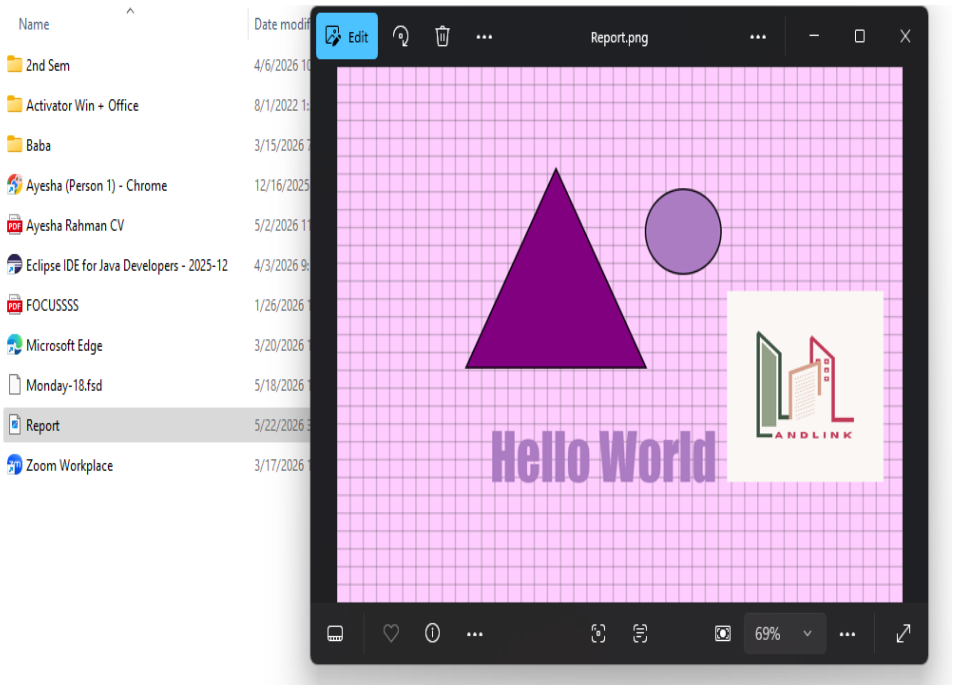

Figure 21 – Exported PNG file viewed in Windows Photos

6.1 Dialogs & Confirmations

To prevent accidental data loss, two confirmation dialogs are implemented. Clicking Clear triggers a confirmation with a reminder that the action can be undone. Navigating Home when unsaved changes exist prompts the user to confirm discarding progress.

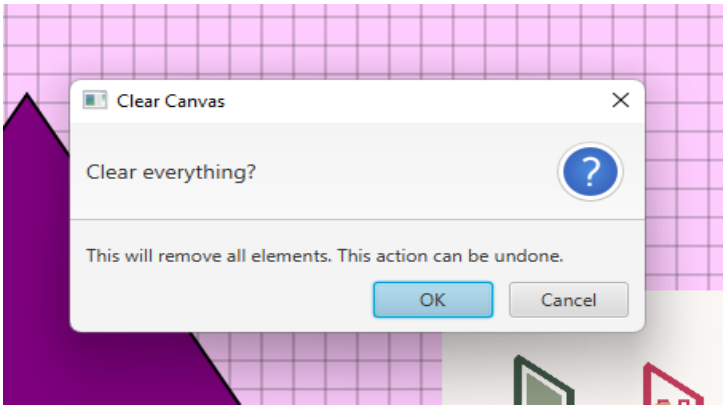

Figure 22 – Clear Canvas confirmation dialog

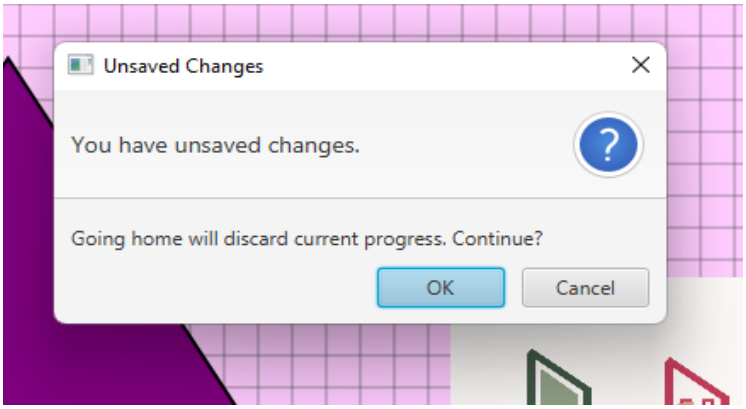

Figure 23 – Unsaved Changes warning when navigating home

7. Command Pattern: Undo / Redo

Every canvas-modifying operation is wrapped in a Command object and executed through CommandManager. This decouples the action from the UI and provides full undo/redo support. The undo stack is a LIFO stack; executing a new command clears the redo stack.

7.1 Command Types

- AddShapeCommand – Adds an element to the canvas; undo removes it.
- DeleteShapeCommand – Removes an element; undo re-adds it.
- MoveShapeCommand – Records old and new positions; undo restores the original.
- ClearCanvasCommand – Snapshots the full element list before clearing; undo restores all elements.

Keyboard shortcuts Ctrl+Z (undo) and Ctrl+Y (redo) are wired directly to CommandManager.undo() and CommandManager.redo() in the scene-level key handler.

8. Complete Source Code

The following section contains the full, unmodified source code for all nineteen Java source files that comprise the FrameStudyo project.

FRAMESTUDYO.java

Main application class. Extends javafx.application.Application. Constructs the entire UI, handles all mouse and keyboard events, wires up the toolbar buttons, and manages transitions between the welcome screen and the main editor.

```
// FRAMESTUDYO.java
package com.framestudyo.framestudyo;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.layout.Priority;
import javafx.scene.layout.GridPane;
import javafx.scene.media.MediaPlayer;
import javafx.scene.paint.Color;
import javafx.stage.FileChooser;
import javafx.stage.Stage;
import java.io.File;
import java.util.Optional;
import javafx.scene.media.Media;
import javafx.scene.media.MediaView;
import javafx.geometry.Pos;
import javafx.geometry.Insets;
import javafx.scene.control.TextInputDialog;

public class FRAMESTUDYO extends Application {
    private String selectedTool = "select";
    private double startX, startY;
    private double dragOffsetX, dragOffsetY;
    private boolean isResizing = false;
    private String resizeHandle = "";
    private double originalX, originalY, originalWidth, originalHeight;
    private Button activeButton = null;
    private String activeBg = "#3e7e9f";
    private String activeHover = "#74d8eb";
    private boolean hasUnsavedChanges = false;
    AlignmentEngine alignmentEngine = new AlignmentEngine();
    ShapeRecognizer shapeRecognizer = new ShapeRecognizer();

    private String btn(String bg, String text) {
        return "-fx-background-color: " + bg + ";" +
                "-fx-text-fill: " + text + ";" +
                "-fx-font-family: 'Segoe UI';" +
                "-fx-font-size: 12px;" +
                "-fx-font-weight: bold;" +
                "-fx-background-radius: 20;" +
                "-fx-padding: 6 14 6 14;" +
                "-fx-cursor: hand;";
    }

    private String activeStyle(String borderColor) {
        return "-fx-background-color: #ffffff;" +
                "-fx-text-fill: #000000;" +
                "-fx-font-family: 'Segoe UI';" +
                "-fx-font-size: 12px;" +
                "-fx-font-weight: bold;" +
                "-fx-background-radius: 20;" +
                "-fx-padding: 6 14 6 14;" +
                "-fx-cursor: hand;" +
                "-fx-border-color: " + borderColor + ";" +
```

```
                "-fx-border-width: 2;" +
                "-fx-border-radius: 20;";
    }

    private Button styledBtn(String label, String bg, String hoverBg) {
        Button b = new Button(label);
        b.setStyle(btn(bg, "#ffffff"));
        b.setOnMouseEntered(e -> { if (b != activeButton) b.setStyle(btn(hoverBg, "#ffffff")); });
        b.setOnMouseExited(e -> { if (b != activeButton) b.setStyle(btn(bg, "#ffffff")); });
        return b;
    }

    private void setActive(Button b, String bg, String hoverBg) {
        if (activeButton != null) {
            final String oldBg = activeBg;
            final String oldHover = activeHover;
            activeButton.setStyle(btn(oldBg, "#ffffff"));
            Button prev = activeButton;
            prev.setOnMouseEntered(e -> { if (prev != activeButton) prev.setStyle(btn(oldHover,
"#ffffff")); });
            prev.setOnMouseExited(e -> { if (prev != activeButton) prev.setStyle(btn(oldBg, "#ffffff"));
});
        }
        activeButton = b;
        activeBg = bg;
        activeHover = hoverBg;
        activeButton.setStyle(activeStyle(bg));
        activeButton.setOnMouseEntered(e -> {});
        activeButton.setOnMouseExited(e -> {});
    }

    private String colorToHex(Color color) {
        return String.format("#%02X%02X%02X",
                (int)(color.getRed() * 255),
                (int)(color.getGreen() * 255),
                (int)(color.getBlue() * 255));
    }

    private void drawSelectionHighlight(Canvas canvas, DesignCanvas designCanvas) {
        if (designCanvas.getSelectedElement() != null) {
            GraphicsContext gc = canvas.getGraphicsContext2D();
            gc.setStroke(Color.web("#3ddcf8"));
            gc.setLineWidth(2);
            gc.setLineDashes(5);
            gc.strokeRect(
                    designCanvas.getSelectedElement().getX() - 4,
                    designCanvas.getSelectedElement().getY() - 4,
                    designCanvas.getSelectedElement().getWidth() + 8,
                    designCanvas.getSelectedElement().getHeight() + 8);
            gc.setLineDashes(0);
        }
    }

    private void drawHandles(Canvas canvas, DesignCanvas designCanvas) {
        if (designCanvas.getSelectedElement() != null) {
            DesignElement el = designCanvas.getSelectedElement();
            GraphicsContext gc = canvas.getGraphicsContext2D();
            double hSize = 8;
            gc.setFill(Color.WHITE);
            gc.setStroke(Color.web("#3ddcf8"));
            gc.setLineWidth(1);
            double[][] handles = {
                    {el.getX() - hSize/2, el.getY() - hSize/2},
                    {el.getX() + el.getWidth() - hSize/2, el.getY() - hSize/2},
                    {el.getX() - hSize/2, el.getY() + el.getHeight() - hSize/2},
                    {el.getX() + el.getWidth() - hSize/2, el.getY() + el.getHeight() - hSize/2}
            };
            for (double[] handle : handles) {
                gc.fillRect(handle[0], handle[1], hSize, hSize);
                gc.strokeRect(handle[0], handle[1], hSize, hSize);
            }
        }
    }
```

```
    private void drawSelection(Canvas canvas, DesignCanvas designCanvas) {
        drawSelectionHighlight(canvas, designCanvas);
        drawHandles(canvas, designCanvas);
    }

    private String getResizeHandle(DesignElement el, double px, double py) {
        double tolerance = 10;
        if (Math.abs(px - el.getX()) < tolerance && Math.abs(py - el.getY()) < tolerance) return "TL";
        if (Math.abs(px - (el.getX() + el.getWidth())) < tolerance && Math.abs(py - el.getY()) <
tolerance) return "TR";
        if (Math.abs(px - el.getX()) < tolerance && Math.abs(py - (el.getY() + el.getHeight())) <
tolerance) return "BL";
        if (Math.abs(px - (el.getX() + el.getWidth())) < tolerance && Math.abs(py - (el.getY() +
el.getHeight())) < tolerance) return "BR";
        return "";
    }

    @Override
    public void start(Stage stage) {
        BorderPane root = new BorderPane();
        root.setStyle("-fx-background-color: transparent;");

        Canvas canvas = new Canvas(860, 600);
        canvas.setStyle("-fx-background-color: transparent;");
        DesignCanvas designCanvas = new DesignCanvas(canvas);
        CommandManager commandManager = new CommandManager();
        designCanvas.setBackgroundColor("#ffffff");
        designCanvas.redraw();

        ScrollPane[] propScrollRef = {null};
        Scene[] mainScene = new Scene[1];

        Media media = new Media(
                getClass().getResource("/com/framestudyo/framestudyo/bg canvas.mp4").toExternalForm());
        MediaPlayer player = new MediaPlayer(media);
        MediaView mediaView = new MediaView(player);
        mediaView.setPreserveRatio(false);
        mediaView.fitWidthProperty().bind(stage.widthProperty());
        mediaView.fitHeightProperty().bind(stage.heightProperty());
        player.setCycleCount(MediaPlayer.INDEFINITE);
        player.play();

        StackPane canvasHolder = new StackPane(canvas);
        canvasHolder.setStyle("-fx-background-color: transparent; -fx-padding: 20;");

        ColorPicker colorPicker = new ColorPicker(Color.web("#ac7cc3"));
        ColorPicker strokeColorPicker = new ColorPicker(Color.BLACK);
        Slider thicknessSlider = new Slider(1, 20, 2);
        thicknessSlider.setShowTickLabels(true);
        thicknessSlider.setMajorTickUnit(5);
        thicknessSlider.setPrefWidth(80);

        MenuButton shapesMenu = new MenuButton("✦ Shapes");
        shapesMenu.setStyle(btn("#eb6891", "#ffffff") + "-fx-background-radius: 20;");
        shapesMenu.setTooltip(new Tooltip("Choose a shape to draw"));

        String[] shapeNames = {"Rectangle", "Circle", "Triangle", "Star", "Arrow", "Line"};
        String[] shapeKeys  = {"rectangle", "circle",  "triangle", "star", "arrow",  "line"};
        for (int i = 0; i < shapeNames.length; i++) {
            final String key  = shapeKeys[i];
            final String name = shapeNames[i];
            MenuItem item = new MenuItem(name);
            item.setOnAction(e -> {
                selectedTool = key;
                shapesMenu.setText("✦ " + name);
                if (activeButton != null) {
                    final String oldBg = activeBg;
                    activeButton.setStyle(btn(oldBg, "#ffffff"));
                    activeButton = null;
                }
```

```
            shapesMenu.setStyle(activeStyle("#eb6891") + "-fx-background-radius: 20;");
        });
        shapesMenu.getItems().add(item);
    }

    // ── Buttons ──────────────────────────────────────────────────────
    Button selectBtn    = styledBtn("⬡ Select",     "#3e7e9f", "#74d8eb");
    Button textBtn      = styledBtn("T Text",        "#ac7cc3", "#e2a9f1");
    Button deleteBtn    = styledBtn("✕ Delete",      "#bc4156", "#fb6c6a");
    Button duplicateBtn = styledBtn("⧉ Duplicate",  "#f19e4f", "#fbbb3e");
    Button clearBtn     = styledBtn("⊘ Clear",       "#bc4156", "#fb6c6a");
    Button undoBtn      = styledBtn("↩ Undo",        "#2b588b", "#3e7e9f");
    Button redoBtn      = styledBtn("↪ Redo",        "#2b588b", "#3e7e9f");
    Button paletteBtn   = styledBtn("🎨 Palette",    "#6a3d8f", "#9b59b6");
    Button analyzeBtn   = styledBtn("📊 Analyze",    "#2b588b", "#3e7e9f");
    Button drawBtn      = styledBtn("✒ Draw",        "#3e8f5a", "#56c97a");

    selectBtn.setTooltip(new Tooltip("Select and move elements"));
    textBtn.setTooltip(new Tooltip("Add text"));
    deleteBtn.setTooltip(new Tooltip("Delete selected element"));
    duplicateBtn.setTooltip(new Tooltip("Duplicate selected element"));
    clearBtn.setTooltip(new Tooltip("Clear entire canvas"));
    undoBtn.setTooltip(new Tooltip("Undo (Ctrl+Z)"));
    redoBtn.setTooltip(new Tooltip("Redo (Ctrl+Y)"));
    paletteBtn.setTooltip(new Tooltip("Smart color palette suggestions"));
    analyzeBtn.setTooltip(new Tooltip("Analyze your design score"));
    drawBtn.setTooltip(new Tooltip("Freehand draw — auto-detects shape"));

    // ── Button actions ──────────────────────────────────────────────
    selectBtn.setOnAction(e -> {
        selectedTool = "select";
        setActive(selectBtn, "#3e7e9f", "#74d8eb");
        shapesMenu.setStyle(btn("#eb6891", "#ffffff") + "-fx-background-radius: 20;");
    });

    textBtn.setOnAction(e -> {
        selectedTool = "text";
        setActive(textBtn, "#ac7cc3", "#e2a9f1");
        shapesMenu.setStyle(btn("#eb6891", "#ffffff") + "-fx-background-radius: 20;");
    });

    drawBtn.setOnAction(e -> {
        selectedTool = "freehand";
        setActive(drawBtn, "#3e8f5a", "#56c97a");
        shapesMenu.setStyle(btn("#eb6891", "#ffffff") + "-fx-background-radius: 20;");
    });

    paletteBtn.setOnAction(e -> SmartColorAdvisor.show(stage, designCanvas));

    analyzeBtn.setOnAction(e ->
            DesignAnalyzer.show(stage, designCanvas, canvas.getWidth(), canvas.getHeight()));

    deleteBtn.setOnAction(e -> {
        if (designCanvas.getSelectedElement() != null) {
            commandManager.executeCommand(
                    new DeleteShapeCommand(designCanvas, designCanvas.getSelectedElement()));
            SessionTracker.getInstance().logAction("Deleted element");
            hasUnsavedChanges = true;
            designCanvas.setSelectedElement(null);
            root.setRight(null);
        }
    });

    duplicateBtn.setOnAction(e -> {
        if (designCanvas.getSelectedElement() != null) {
            DesignElement original = designCanvas.getSelectedElement();
            DesignElement copy = original.duplicate();
            copy.setX(original.getX() + 20);
            copy.setY(original.getY() + 20);
            commandManager.executeCommand(new AddShapeCommand(designCanvas, copy));
```

```
            SessionTracker.getInstance().logAction("Duplicated element");
            hasUnsavedChanges = true;
        }
    });

    clearBtn.setOnAction(e -> {
        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle("Clear Canvas");
        alert.setHeaderText("Clear everything?");
        alert.setContentText("This will remove all elements. This action can be undone.");
        alert.showAndWait().ifPresent(response -> {
            if (response == ButtonType.OK) {
                commandManager.executeCommand(new ClearCanvasCommand(designCanvas));
                SessionTracker.getInstance().logAction("Cleared canvas");
                designCanvas.setSelectedElement(null);
                root.setRight(null);
            }
        });
    });

    undoBtn.setOnAction(e -> {
        commandManager.undo();
        SessionTracker.getInstance().logAction("Undo");
        designCanvas.redraw();
    });

    redoBtn.setOnAction(e -> {
        commandManager.redo();
        SessionTracker.getInstance().logAction("Redo");
        designCanvas.redraw();
    });

    // ── Status bar ──────────────────────────────────────────────
    Label statusLabel = new Label("X: 0 Y: 0 | Tool: Select | Selected: None");
    statusLabel.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; " +
            "-fx-padding: 5 12 5 12; -fx-font-size: 11px;");
    // ── Circular Stats button (bottom-left) ─────────────────────
    Button statsBtn = new Button("✦");
    statsBtn.setTooltip(new Tooltip("Session stats & history"));
    String statsBtnNormal =
            "-fx-background-color: #ac7cc3;" +
                    "-fx-text-fill: white;" +
                    "-fx-font-size: 15px;" +
                    "-fx-font-weight: bold;" +
                    "-fx-background-radius: 50%;" +
                    "-fx-min-width: 32px; -fx-min-height: 32px;" +
                    "-fx-max-width: 32px; -fx-max-height: 32px;" +
                    "-fx-padding: 0; -fx-cursor: hand;";
    String statsBtnHover =
            "-fx-background-color: #3ddcf8;" +
                    "-fx-text-fill: #1a1a2e;" +
                    "-fx-font-size: 15px;" +
                    "-fx-font-weight: bold;" +
                    "-fx-background-radius: 50%;" +
                    "-fx-min-width: 32px; -fx-min-height: 32px;" +
                    "-fx-max-width: 32px; -fx-max-height: 32px;" +
                    "-fx-padding: 0; -fx-cursor: hand;";
    statsBtn.setStyle(statsBtnNormal);
    statsBtn.setOnMouseEntered(e -> statsBtn.setStyle(statsBtnHover));
    statsBtn.setOnMouseExited(e  -> statsBtn.setStyle(statsBtnNormal));

    Region statusSpacer = new Region();
    HBox.setHgrow(statusSpacer, Priority.ALWAYS);

    HBox statusBar = new HBox(6);
    statusBar.setAlignment(Pos.CENTER_LEFT);
    statusBar.setPadding(new Insets(4, 8, 4, 8));
    statusBar.setStyle("-fx-background-color: rgba(43,70,114,0.75);");
    statusBar.getChildren().addAll(statsBtn, statusLabel, statusSpacer);

    canvas.setOnMouseMoved(e -> {
        String selected = designCanvas.getSelectedElement() != null ?
```

```
                    designCanvas.getSelectedElement().getClass().getSimpleName().replace("Element", "")
: "None";
            statusLabel.setText("X: " + (int)e.getX() + " Y: " + (int)e.getY() +
                    " | Tool: " + selectedTool.substring(0, 1).toUpperCase() + selectedTool.substring(1)
+
                    " | Selected: " + selected);
        });

        // ── Mouse pressed ──────────────────────────────────────────
        canvas.setOnMousePressed(e -> {
            startX = e.getX();
            startY = e.getY();

            // Clear snap guides on every new press
            alignmentEngine.clearGuides(canvas, designCanvas);

            if (e.getClickCount() == 2 &&
                    designCanvas.getSelectedElement() instanceof TextElement txt) {

                TextInputDialog dialog =
                        new TextInputDialog(txt.getText());

                dialog.setTitle("Edit Text");

                dialog.setHeaderText(null);

                dialog.setContentText("Text:");

                dialog.showAndWait().ifPresent(newText -> {

                    txt.setText(newText);

                    hasUnsavedChanges = true;

                    designCanvas.redraw();

                    drawSelection(canvas, designCanvas);
                });
            }

            if (selectedTool.equals("freehand")) {
                shapeRecognizer.startStroke(startX, startY);
                return;
            }

            if (selectedTool.equals("select")) {
                isResizing = false;
                if (designCanvas.getSelectedElement() != null) {
                    resizeHandle = getResizeHandle(designCanvas.getSelectedElement(), startX, startY);
                    if (!resizeHandle.isEmpty()) {
                        isResizing = true;
                        DesignElement el = designCanvas.getSelectedElement();
                        originalX      = el.getX();
                        originalY      = el.getY();
                        originalWidth  = el.getWidth();
                        originalHeight = el.getHeight();
                        return;
                    }
                }
                designCanvas.setSelectedElement(null);
                root.setRight(null);
                java.util.List<DesignElement> elements = designCanvas.getElements();
                for (int i = elements.size() - 1; i >= 0; i--) {
                    if (elements.get(i).contains(startX, startY)) {
                        designCanvas.setSelectedElement(elements.get(i));
                        dragOffsetX = startX - designCanvas.getSelectedElement().getX();
                        dragOffsetY = startY - designCanvas.getSelectedElement().getY();
                        root.setRight(propScrollRef[0]);
                        break;
```

```
                }
            }
            designCanvas.redraw();
            drawSelection(canvas, designCanvas);
        }
    });

    // ── Mouse dragged ──────────────────────────────────────────
    canvas.setOnMouseDragged(e -> {
        double endX = e.getX();
        double endY = e.getY();

        if (selectedTool.equals("freehand")) {
            shapeRecognizer.addPoint(endX, endY);
            designCanvas.redraw();
            shapeRecognizer.drawStroke(canvas.getGraphicsContext2D());
            return;
        }

        if (selectedTool.equals("select")) {
            if (isResizing && designCanvas.getSelectedElement() != null) {
                DesignElement el = designCanvas.getSelectedElement();
                double dx = endX - startX, dy = endY - startY;
                switch (resizeHandle) {
                    case "BR":
                        el.setWidth(Math.max(10, originalWidth + dx));
                        el.setHeight(Math.max(10, originalHeight + dy));
                        break;
                    case "TR":
                        el.setWidth(Math.max(10, originalWidth + dx));
                        el.setY(originalY + dy);
                        el.setHeight(Math.max(10, originalHeight - dy));
                        break;
                    case "BL":
                        el.setX(originalX + dx);
                        el.setWidth(Math.max(10, originalWidth - dx));
                        el.setHeight(Math.max(10, originalHeight + dy));
                        break;
                    case "TL":
                        el.setX(originalX + dx);
                        el.setY(originalY + dy);
                        el.setWidth(Math.max(10, originalWidth - dx));
                        el.setHeight(Math.max(10, originalHeight - dy));
                        break;
                }
                designCanvas.redraw();
                drawSelection(canvas, designCanvas);
            } else if (designCanvas.getSelectedElement() != null) {
                // ── Smart snap drag ──────────────────────────────
                DesignElement dragEl = designCanvas.getSelectedElement();
                double[] snapped = alignmentEngine.snap(
                        dragEl,
                        endX - dragOffsetX,
                        endY - dragOffsetY,
                        designCanvas.getElements());
                dragEl.setX(snapped[0]);
                dragEl.setY(snapped[1]);
                designCanvas.redraw();
                alignmentEngine.drawGuides(canvas, designCanvas);
                drawSelection(canvas, designCanvas);
            }
        } else {
            // Shape preview while dragging
            double width  = Math.abs(endX - startX);
            double height = Math.abs(endY - startY);
            double x = Math.min(startX, endX);
            double y = Math.min(startY, endY);
            designCanvas.redraw();
            GraphicsContext gc = canvas.getGraphicsContext2D();
            gc.setStroke(Color.web("#3ddcf8"));
            gc.setLineWidth(1.5);
            gc.setLineDashes(6);
            switch (selectedTool) {
                case "rectangle":
```

```
                    gc.strokeRect(x, y, width, height);
                    break;
                case "circle":
                    gc.strokeOval(x, y, width, height);
                    break;
                case "line":
                    gc.strokeLine(startX, startY, endX, endY);
                    break;
                case "text":
                    gc.strokeRect(x, y, 200, 50);
                    break;
                case "triangle":
                    double[] px = {x + width/2, x, x + width};
                    double[] py = {y, y + height, y + height};
                    gc.strokePolygon(px, py, 3);
                    break;
                case "star":
                    double cx = x + width/2, cy = y + height/2;
                    double oR = Math.min(width, height)/2, iR = oR * 0.4;
                    double[] sx = new double[10], sy = new double[10];
                    for (int i = 0; i < 10; i++) {
                        double angle = Math.PI/ 5 * i - Math.PI/ 2;
                        double r = (i % 2 == 0) ? oR : iR;
                        sx[i] = cx + r * Math.cos(angle);
                        sy[i] = cy + r * Math.sin(angle);
                    }
                    gc.strokePolygon(sx, sy, 10);
                    break;
                case "arrow":
                    double sH = height * 0.4, sY = y + height * 0.3, hW = width * 0.4;
                    double[] ax = {x, x+width-hW, x+width-hW, x+width, x+width-hW, x+width-hW, x};
                    double[] ay = {sY, sY, y, y+height/2, y+height, sY+sH, sY+sH};
                    gc.strokePolygon(ax, ay, 7);
                    break;
            }
            gc.setLineDashes(0);
        }
    });

    // ── Mouse released ──────────────────────────────────────────
    canvas.setOnMouseReleased(e -> {
        // Clear snap guides on release
        alignmentEngine.clearGuides(canvas, designCanvas);

        if (selectedTool.equals("select")) {
            isResizing = false;
            return;
        }

        double endX   = e.getX(), endY = e.getY();
        double width  = Math.abs(endX - startX), height = Math.abs(endY - startY);
        double x = Math.min(startX, endX),        y = Math.min(startY, endY);

        DesignElement element = null;

        switch (selectedTool) {
            case "freehand":
                DesignElement recognized = shapeRecognizer.recognize(
                        colorToHex(colorPicker.getValue()),
                        colorToHex(strokeColorPicker.getValue()),
                        thicknessSlider.getValue());
                if (recognized != null) {
                    commandManager.executeCommand(new AddShapeCommand(designCanvas, recognized));
                    SessionTracker.getInstance().logAction(
                            "Drew (recognized as " + shapeRecognizer.lastRecognizedLabel() + ")");
                    hasUnsavedChanges = true;
                    statusLabel.setText("✦ Recognized: " +
                            shapeRecognizer.lastRecognizedLabel().toUpperCase() +
                            " — click Select to move it");
                }
                designCanvas.redraw();
                return;
```

```
                case "rectangle":
                    element = new RectangleElement(x, y, width, height);
                    element.setFillColor(colorToHex(colorPicker.getValue()));
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "circle":
                    element = new CircleElement(x, y, width, height);
                    element.setFillColor(colorToHex(colorPicker.getValue()));
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "triangle":
                    element = new TriangleElement(x, y, width, height);
                    element.setFillColor(colorToHex(colorPicker.getValue()));
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "star":
                    element = new StarElement(x, y, width, height);
                    element.setFillColor(colorToHex(colorPicker.getValue()));
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "arrow":
                    element = new ArrowElement(x, y, width, height);
                    element.setFillColor(colorToHex(colorPicker.getValue()));
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "line":
                    element = new LineElement(startX, startY, endX, endY);
                    element.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                    element.setStrokeWidth(thicknessSlider.getValue());
                    break;
                case "text":
                    Dialog<String[]> dialog = new Dialog<>();
                    dialog.setTitle("Add Text");
                    dialog.setHeaderText("Enter your text details");
                    ButtonType okButton = new ButtonType("Add", ButtonBar.ButtonData.OK_DONE);
                    dialog.getDialogPane().getButtonTypes().addAll(okButton, ButtonType.CANCEL);
                    GridPane grid = new GridPane();
                    grid.setHgap(10);
                    grid.setVgap(10);
                    TextField textInput = new TextField("Your text here");
                    ComboBox<String> fontPicker = new ComboBox<>();
                    fontPicker.getItems().addAll("Arial", "Times New Roman", "Verdana",
                            "Georgia", "Courier New", "Comic Sans MS", "Impact", "Trebuchet MS");
                    fontPicker.setValue("Arial");
                    ComboBox<String> sizePicker = new ComboBox<>();
                    sizePicker.getItems().addAll("12", "16", "20", "24", "32", "40", "48", "64", "72");
                    sizePicker.setValue("24");
                    grid.add(new Label("Text:"), 0, 0);
                    grid.add(textInput, 1, 0);
                    grid.add(new Label("Font:"), 0, 1);
                    grid.add(fontPicker, 1, 1);
                    grid.add(new Label("Size:"), 0, 2);
                    grid.add(sizePicker, 1, 2);
                    dialog.getDialogPane().setContent(grid);
                    dialog.setResultConverter(db -> db == okButton ?
                            new String[]{textInput.getText(), fontPicker.getValue(),
sizePicker.getValue()} : null);
                    Optional<String[]> result = dialog.showAndWait();
                    result.ifPresent(values -> {
                        TextElement te = new TextElement(x, y, values[0]);
                        te.setFillColor(colorToHex(colorPicker.getValue()));
                        te.setStrokeColor(colorToHex(strokeColorPicker.getValue()));
                        te.setFontFamily(values[1]);
                        te.setFontSize(Double.parseDouble(values[2]));
                        commandManager.executeCommand(new AddShapeCommand(designCanvas, te));
                        SessionTracker.getInstance().logAction("Added Text");
                        hasUnsavedChanges = true;
                    });
                    return;
```

```
            }

            if (element != null) {
                commandManager.executeCommand(new AddShapeCommand(designCanvas, element));
                SessionTracker.getInstance().logAction("Added " +
                        element.getClass().getSimpleName().replace("Element", ""));
                hasUnsavedChanges = true;
            }
        });

        // ── Properties Panel ──────────────────────────────────────
        ColorPicker propFillPicker     = new ColorPicker(Color.web("#ac7cc3"));
        ColorPicker propStrokePicker   = new ColorPicker(Color.BLACK);
        Slider propThicknessSlider     = new Slider(1, 20, 2);
        propThicknessSlider.setShowTickLabels(true);
        propThicknessSlider.setMajorTickUnit(5);
        Slider propOpacitySlider = new Slider(0.0, 1.0, 1.0);
        propOpacitySlider.setShowTickLabels(true);
        propOpacitySlider.setMajorTickUnit(0.5);
        Slider propRotationSlider = new Slider(0, 360, 0);
        propRotationSlider.setShowTickLabels(true);
        propRotationSlider.setMajorTickUnit(90);

        propFillPicker.setOnAction(e -> {
            if (designCanvas.getSelectedElement() != null) {
                designCanvas.updateSelectedFillColor(colorToHex(propFillPicker.getValue()));
                drawSelection(canvas, designCanvas);
            }
        });
        propStrokePicker.setOnAction(e -> {
            if (designCanvas.getSelectedElement() != null) {
                designCanvas.updateSelectedStrokeColor(colorToHex(propStrokePicker.getValue()));
                drawSelection(canvas, designCanvas);
            }
        });
        propThicknessSlider.valueProperty().addListener((obs, o, n) -> {
            if (designCanvas.getSelectedElement() != null) {
                designCanvas.updateSelectedStrokeWidth(n.doubleValue());
                drawSelection(canvas, designCanvas);
            }
        });
        propOpacitySlider.valueProperty().addListener((obs, o, n) -> {
            if (designCanvas.getSelectedElement() != null) {
                designCanvas.getSelectedElement().setOpacity(n.doubleValue());
                designCanvas.redraw();
                drawSelection(canvas, designCanvas);
            }
        });
        propRotationSlider.valueProperty().addListener((obs, o, n) -> {
            if (designCanvas.getSelectedElement() != null) {
                designCanvas.getSelectedElement().setAngle(n.doubleValue());
                designCanvas.redraw();
                drawSelection(canvas, designCanvas);
            }
        });

        Button gridToggleBtn = styledBtn("Show Grid", "#3e7e9f", "#74d8eb");
        gridToggleBtn.setMaxWidth(Double.MAX_VALUE);
        gridToggleBtn.setOnAction(e -> {
            boolean ns = !designCanvas.isShowGrid();
            designCanvas.setShowGrid(ns);
            gridToggleBtn.setText(ns ? "Hide Grid" : "Show Grid");
        });

        Button bringToFrontBtn = styledBtn("Bring to Front", "#2b588b", "#3e7e9f");
        Button bringForwardBtn = styledBtn("Bring Forward",  "#2b588b", "#3e7e9f");
        Button sendBackwardBtn = styledBtn("Send Backward",  "#2b4672", "#2b588b");
        Button sendToBackBtn   = styledBtn("Send to Back",   "#2b4672", "#2b588b");
        bringToFrontBtn.setMaxWidth(Double.MAX_VALUE);
        bringForwardBtn.setMaxWidth(Double.MAX_VALUE);
        sendBackwardBtn.setMaxWidth(Double.MAX_VALUE);
        sendToBackBtn.setMaxWidth(Double.MAX_VALUE);
```

```
        bringForwardBtn.setOnAction(e -> { designCanvas.bringForward();  drawSelection(canvas,
designCanvas); });
        sendBackwardBtn.setOnAction(e -> { designCanvas.sendBackward();  drawSelection(canvas,
designCanvas); });
        bringToFrontBtn.setOnAction(e -> { designCanvas.bringToFront();  drawSelection(canvas,
designCanvas); });
        sendToBackBtn.setOnAction(e ->   { designCanvas.sendToBack();    drawSelection(canvas,
designCanvas); });

        String labelStyle = "-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; -fx-font-weight: bold;
-fx-font-size: 12px;";
        String titleStyle = "-fx-text-fill: #3ddcf8; -fx-font-family: 'Segoe UI'; -fx-font-weight: bold;
-fx-font-size: 14px;";

        Label propTitle    = new Label("✦ Properties"); propTitle.setStyle(titleStyle);
        Label fillLbl      = new Label("Fill Color:");   fillLbl.setStyle(labelStyle);
        Label strokeLbl    = new Label("Stroke Color:"); strokeLbl.setStyle(labelStyle);
        Label thickLbl     = new Label("Stroke Width:"); thickLbl.setStyle(labelStyle);
        Label opacityLbl   = new Label("Opacity:");      opacityLbl.setStyle(labelStyle);
        Label rotationLbl  = new Label("Rotation:");     rotationLbl.setStyle(labelStyle);
        Label layersLbl    = new Label("✦ Layers:");    layersLbl.setStyle(titleStyle);
        Label gridLbl      = new Label("✦ Canvas:");    gridLbl.setStyle(titleStyle);

        VBox propertiesPanel = new VBox(10);
        propertiesPanel.setStyle("-fx-background-color: #471f3f; -fx-padding: 15;");
        propertiesPanel.setPrefWidth(200);
        propertiesPanel.setMinWidth(200);
        propertiesPanel.getChildren().addAll(
                propTitle, new Separator(),
                fillLbl, propFillPicker,
                strokeLbl, propStrokePicker,
                thickLbl, propThicknessSlider,
                opacityLbl, propOpacitySlider,
                rotationLbl, propRotationSlider,
                new Separator(),
                gridLbl, gridToggleBtn,
                new Separator(),
                layersLbl,
                bringToFrontBtn, bringForwardBtn,
                sendBackwardBtn, sendToBackBtn
        );

        propScrollRef[0] = new ScrollPane(propertiesPanel);
        propScrollRef[0].setFitToWidth(true);
        propScrollRef[0].setPrefWidth(220);
        propScrollRef[0].setStyle("-fx-background-color: #471f3f; -fx-background: #471f3f;");

        // ── File menu ──────────────────────────────────────────────
        MenuButton fileMenu = new MenuButton(" File");
        fileMenu.setStyle(btn("#3ddcf8", "#ffffff") + "-fx-background-radius: 20;");
        fileMenu.setTooltip(new Tooltip("File operations"));

        MenuItem homeItem   = new MenuItem(" Home");
        MenuItem saveItem   = new MenuItem(" Save");
        MenuItem loadItem   = new MenuItem(" Load");
        MenuItem exportItem = new MenuItem("↑ Export PNG");

        homeItem.setOnAction(e -> {
            if (hasUnsavedChanges) {
                Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
                alert.setTitle("Unsaved Changes");
                alert.setHeaderText("You have unsaved changes.");
                alert.setContentText("Going home will discard current progress. Continue?");
                Optional<ButtonType> result = alert.showAndWait();
                if (result.isEmpty() || result.get() != ButtonType.OK) return;
            }
            //SessionTracker.getInstance().pauseSession();
            player.pause();

            stage.setScene(
                    WelcomeScreen.create(
```

```
                        stage,

                        () -> {

                            designCanvas.getElements().clear();

                            designCanvas.setSelectedElement(null);

                            selectedTool = "select";

                            root.setRight(null);

                            hasUnsavedChanges = false;

                            designCanvas.redraw();
                           // SessionTracker.getInstance().resumeSession();
                            player.play();
                            stage.setScene(mainScene[0]);
                        },

                        (loaded) -> {

                            designCanvas.getElements().clear();

                            designCanvas.getElements().addAll(loaded);

                            designCanvas.setSelectedElement(null);

                            designCanvas.redraw();
                          //  SessionTracker.getInstance().resumeSession();
                            player.play();
                            stage.setScene(mainScene[0]);
                        }
                )
        );
    });

    saveItem.setOnAction(e -> {

        FileChooser fc = new FileChooser();

        fc.setTitle("Save Design");

        fc.getExtensionFilters().add(
                new FileChooser.ExtensionFilter(
                        "FrameStudyo Files",
                        "*.fsd"
                )
        );

        File file = fc.showSaveDialog(stage);

        if (file != null) {

            FileManager.save(
                    designCanvas.getElements(),
                    file.getAbsolutePath()
            );

            Alert success = new Alert(Alert.AlertType.INFORMATION);

            success.setTitle("Saved!");

            success.setHeaderText(null);
```

```
                success.setContentText("Design saved successfully!");

                success.showAndWait();

                hasUnsavedChanges = false;

                SessionTracker.getInstance()
                        .logAction("Saved design");
            }
        });

        loadItem.setOnAction(e -> {
            FileChooser fc = new FileChooser();
            fc.setTitle("Load Design");
            fc.getExtensionFilters().add(new FileChooser.ExtensionFilter("FrameStudyo Files", "*.fsd"));
            File file = fc.showOpenDialog(stage);
            if (file != null) {
                java.util.List<DesignElement> loaded = FileManager.load(file.getAbsolutePath());
                if (loaded != null) {
                    designCanvas.getElements().clear();
                    designCanvas.getElements().addAll(loaded);
                    SessionTracker.getInstance().logAction("Loaded design from file");
                    designCanvas.redraw();
                }
            }
        });

        exportItem.setOnAction(e -> {
            FileChooser fc = new FileChooser();
            fc.setTitle("Export as PNG");
            fc.getExtensionFilters().add(new FileChooser.ExtensionFilter("PNG Image", "*.png"));
            File file = fc.showSaveDialog(stage);
            if (file != null) {
                javafx.scene.image.WritableImage image = new javafx.scene.image.WritableImage(
                        (int)canvas.getWidth(), (int)canvas.getHeight());
                canvas.snapshot(null, image);
                try {
                    java.awt.image.BufferedImage bi = new java.awt.image.BufferedImage(
                            (int)canvas.getWidth(), (int)canvas.getHeight(),
                            java.awt.image.BufferedImage.TYPE_INT_ARGB);
                    for (int iy = 0; iy < (int)canvas.getHeight(); iy++)
                        for (int ix = 0; ix < (int)canvas.getWidth(); ix++) {
                            javafx.scene.paint.Color fx = image.getPixelReader().getColor(ix, iy);
                            bi.setRGB(ix, iy, new java.awt.Color(
                                    (float)fx.getRed(), (float)fx.getGreen(),
                                    (float)fx.getBlue(), (float)fx.getOpacity()).getRGB());
                        }
                    javax.imageio.ImageIO.write(bi, "PNG", file);
                    Alert success = new Alert(Alert.AlertType.INFORMATION);
                    success.setTitle("Export Successful");
                    success.setHeaderText(null);
                    success.setContentText("Design exported successfully as PNG!");
                    success.showAndWait();
                    SessionTracker.getInstance().logAction("Exported as PNG");
                } catch (java.io.IOException ex) {
                    System.out.println("Export failed: " + ex.getMessage());
                }
            }
        });

        fileMenu.getItems().addAll(homeItem, saveItem, loadItem, exportItem);

        // ── Top toolbar extras ──────────────────────────────────
        Label bgLabel = new Label("Canvas BG:");
        bgLabel.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; -fx-font-weight: bold;");
        ColorPicker topBgPicker = new ColorPicker(Color.WHITE);
        topBgPicker.setTooltip(new Tooltip("Change canvas background color"));
        topBgPicker.setOnAction(e ->
designCanvas.setBackgroundColor(colorToHex(topBgPicker.getValue())));
```

```
        Button topGridBtn = styledBtn("⊞ Grid", "#3e7e9f", "#74d8eb");
        topGridBtn.setTooltip(new Tooltip("Toggle canvas grid"));
        topGridBtn.setOnAction(e -> {
            boolean ns = !designCanvas.isShowGrid();
            designCanvas.setShowGrid(ns);
            topGridBtn.setText(ns ? "⊞ Hide Grid" : "⊞ Grid");
        });

        Button importBtn = styledBtn("🖼 Image", "#2b588b", "#3e7e9f");
        importBtn.setOnAction(e -> {
            FileChooser fc = new FileChooser();
            fc.setTitle("Import Image");
            fc.getExtensionFilters().add(
                    new FileChooser.ExtensionFilter("Image Files", "*.png", "*.jpg", "*.jpeg",
"*.gif"));
            File file = fc.showOpenDialog(stage);
            if (file != null) {
                ImageElement img = new ImageElement(100, 100, 200, 200, file.getAbsolutePath());
                commandManager.executeCommand(new AddShapeCommand(designCanvas, img));
                SessionTracker.getInstance().logAction("Imported image");
                hasUnsavedChanges = true;
            }
        });

        // ── Toolbar ──────────────────────────────────────────────────────
        ToolBar toolbar = new ToolBar(
                fileMenu, new Separator(),
                shapesMenu, selectBtn, textBtn, drawBtn, new Separator(), importBtn,
                deleteBtn, duplicateBtn, clearBtn, new Separator(),
                undoBtn, redoBtn, new Separator(),
                paletteBtn, analyzeBtn, new Separator(),
                bgLabel, topBgPicker, topGridBtn
        );
        toolbar.setStyle("-fx-background-color: rgba(43,70,114,0.75); -fx-padding: 6 10 6 10;");

        VBox toolbars = new VBox(toolbar);
        toolbars.setStyle("-fx-background-color: transparent;");

        root.setTop(toolbars);
        root.setCenter(canvasHolder);
        root.setRight(null);
        // ── Stats panel action ──────────────────────────────────────────
        statsBtn.setOnAction(e -> {
            SessionTracker.SessionStats stats =
                    SessionTracker.getInstance().computeStats(designCanvas);
            java.util.List<SessionTracker.ActionEntry> log =
                    SessionTracker.getInstance().getRecentHistory(12);

            Stage panel = new Stage();
            panel.initOwner(stage);
            panel.initModality(javafx.stage.Modality.NONE);
            panel.initStyle(javafx.stage.StageStyle.UNDECORATED);

            Label hdr = new Label("✦  Stats & History");
            hdr.setStyle("-fx-text-fill: #3ddcf8; -fx-font-family: 'Segoe UI';" +
                    "-fx-font-weight: bold; -fx-font-size: 15px;");

            VBox timeTile = statTile("🕒 Time", stats.sessionTime);

            Label timeLabel =
                    (Label) timeTile.getChildren().get(0);

            HBox tiles = new HBox(8,
                    timeTile,
                    statTile("✦ Elements", String.valueOf(stats.totalElements)),
                    statTile("📋 Actions", String.valueOf(stats.totalActions)),
                    statTile("📐 Coverage", String.format("%.0f%%", stats.coveragePercent))
            );
            tiles.setAlignment(Pos.CENTER_LEFT);
```

```
            javafx.animation.Timeline timerUpdater =
                    new javafx.animation.Timeline(

                            new javafx.animation.KeyFrame(
                                    javafx.util.Duration.seconds(1),

                                    ev -> {

                                        long elapsed =
                                                SessionTracker.getInstance()
                                                        .elapsedMs();

                                        long minutes = elapsed / 60000;

                                        long seconds =
                                                (elapsed % 60000) / 1000;

                                        timeLabel.setText(
                                                minutes + "m " + seconds + "s"
                                        );
                                    }
                            )
                    );

            timerUpdater.setCycleCount(
                    javafx.animation.Animation.INDEFINITE
            );

            timerUpdater.play();

            Label shapesHdr = new Label("Shapes used:");
            shapesHdr.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI';" +
                    "-fx-font-weight: bold; -fx-font-size: 11px;");
            HBox shapeChips = new HBox(6);
            shapeChips.setAlignment(Pos.CENTER_LEFT);
            if (stats.shapeCounts.isEmpty()) {
                Label none = new Label("No shapes yet");
                none.setStyle("-fx-text-fill: #666; -fx-font-size: 11px;");
                shapeChips.getChildren().add(none);
            } else {
                stats.shapeCounts.entrySet().stream()
                        .sorted(java.util.Map.Entry.<String,Long>comparingByValue().reversed())
                        .forEach(entry -> {
                            Label chip = new Label(entry.getKey() + " x" + entry.getValue());
                            chip.setStyle("-fx-background-color: #2b2b4a; -fx-text-fill: #3ddcf8;" +
                                    "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                                    "-fx-background-radius: 10; -fx-padding: 3 8 3 8;");
                            shapeChips.getChildren().add(chip);
                        });
            }

            Label colorLbl = new Label("Top color:");
            colorLbl.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI';" +
                    "-fx-font-size: 11px; -fx-font-weight: bold;");
            javafx.scene.shape.Rectangle swatch = new javafx.scene.shape.Rectangle(22, 22);
            swatch.setArcWidth(6); swatch.setArcHeight(6);
            try { swatch.setFill(Color.web(stats.mostUsedColor)); }
            catch (Exception ex) { swatch.setFill(Color.GRAY); }
            swatch.setStroke(Color.web("#3ddcf8")); swatch.setStrokeWidth(1);
            Label colorHex = new Label(stats.mostUsedColor);
            colorHex.setStyle("-fx-text-fill: #ccc; -fx-font-family: 'Segoe UI'; -fx-font-size: 11px;");
            HBox colorRow = new HBox(8, colorLbl, swatch, colorHex);
            colorRow.setAlignment(Pos.CENTER_LEFT);

            Label histHdr = new Label("Recent actions:");
            histHdr.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI';" +
                    "-fx-font-weight: bold; -fx-font-size: 11px;");
            VBox histList = new VBox(3);
            histList.setStyle("-fx-background-color: #12102a; -fx-background-radius: 8; -fx-padding:
8;");
            java.util.List<SessionTracker.ActionEntry> reversed = new java.util.ArrayList<>(log);
```

```
            java.util.Collections.reverse(reversed);
            for (SessionTracker.ActionEntry entry : reversed) {
                Label row = new Label(entry.toString());
                row.setStyle("-fx-text-fill: #aaa; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");
                row.setMaxWidth(Double.MAX_VALUE);
                histList.getChildren().add(row);
            }
            if (log.isEmpty()) {
                Label empty = new Label("No actions recorded yet.");
                empty.setStyle("-fx-text-fill: #555; -fx-font-size: 10px;");
                histList.getChildren().add(empty);
            }
            ScrollPane histScroll = new ScrollPane(histList);
            histScroll.setFitToWidth(true);
            histScroll.setPrefHeight(160);
            histScroll.setStyle("-fx-background-color: transparent; -fx-background: transparent;");
            histScroll.setHbarPolicy(ScrollPane.ScrollBarPolicy.NEVER);
            histScroll.setVbarPolicy(ScrollPane.ScrollBarPolicy.AS_NEEDED);

            Label startLbl = new Label("Session started: " + stats.sessionStart);
            startLbl.setStyle("-fx-text-fill: #555; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");

            Button closeBtn = new Button("Close");
            closeBtn.setStyle("-fx-background-color: #ac7cc3; -fx-text-fill: white;" +
                    "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                    "-fx-background-radius: 20; -fx-padding: 5 18 5 18; -fx-cursor: hand;");
            closeBtn.setOnMouseEntered(ev -> closeBtn.setStyle(
                    "-fx-background-color: #3ddcf8; -fx-text-fill: #1a1a2e;" +
                            "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                            "-fx-background-radius: 20; -fx-padding: 5 18 5 18; -fx-cursor: hand;"));
            closeBtn.setOnMouseExited(ev -> closeBtn.setStyle(
                    "-fx-background-color: #ac7cc3; -fx-text-fill: white;" +
                            "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                            "-fx-background-radius: 20; -fx-padding: 5 18 5 18; -fx-cursor: hand;"));
            closeBtn.setOnAction(ev -> {

                timerUpdater.stop();

                panel.close();
            });
            HBox closeRow = new HBox(closeBtn);
            closeRow.setAlignment(Pos.CENTER_RIGHT);

            VBox panelRoot = new VBox(12,
                    hdr, new Separator(),
                    tiles, new Separator(),
                    shapesHdr, shapeChips,
                    colorRow, new Separator(),
                    histHdr, histScroll,
                    startLbl, closeRow
            );
            panelRoot.setPadding(new Insets(18));
            panelRoot.setPrefWidth(380);
            panelRoot.setStyle(
                    "-fx-background-color: #1e1235;" +
                            "-fx-border-color: #ac7cc3;" +
                            "-fx-border-width: 1.5;" +
                            "-fx-border-radius: 14;" +
                            "-fx-background-radius: 14;");

            panel.setScene(new javafx.scene.Scene(panelRoot));
            panel.show();
            panel.setX(stage.getX() + 12);
            panel.setY(stage.getY() + stage.getHeight() - 520);
        });

        root.setBottom(statusBar);

        StackPane mainLayer = new StackPane(mediaView, root);
        Scene scene = new Scene(mainLayer, 1200, 750);
        mainScene[0] = scene;
```

```
        // ── Keyboard shortcuts ──────────────────────────────────────
        scene.setOnKeyPressed(e -> {
            switch (e.getCode()) {
                case Z:
                    if (e.isControlDown()) { commandManager.undo(); designCanvas.redraw(); }
                    break;
                case Y:
                    if (e.isControlDown()) { commandManager.redo(); designCanvas.redraw(); }
                    break;
                case S:
                    if (e.isControlDown()) {
                        FileChooser fc = new FileChooser();
                        fc.getExtensionFilters().add(
                                new FileChooser.ExtensionFilter("FrameStudyo Files", "*.fsd"));
                        File file = fc.showSaveDialog(stage);
                        if (file != null) FileManager.save(designCanvas.getElements(),
file.getAbsolutePath());
                    }
                    break;
                case BACK_SPACE:
                    if (designCanvas.getSelectedElement() != null) {
                        commandManager.executeCommand(
                                new DeleteShapeCommand(designCanvas,
designCanvas.getSelectedElement()));
                        SessionTracker.getInstance().logAction("Deleted element (backspace)");
                        hasUnsavedChanges = true;
                        designCanvas.setSelectedElement(null);
                        root.setRight(null);
                    }
                    break;
                default:
                    break;
            }
        });

        // ── Welcome screen ──────────────────────────────────────────
        setActive(selectBtn, "#3e7e9f", "#74d8eb");

        stage.setTitle("FrameStudyo");
        stage.setMinWidth(900);
        stage.setMinHeight(650);
        stage.setMaximized(true);
        stage.setScene(

                WelcomeScreen.create(

                        stage,

                        () -> {

                            designCanvas.getElements().clear();

                            designCanvas.setSelectedElement(null);

                            selectedTool = "select";

                            root.setRight(null);

                            hasUnsavedChanges = false;

                            designCanvas.redraw();

                            stage.setScene(mainScene[0]);
                            player.play();
                        },

                        (loaded) -> {
```

```
                            designCanvas.getElements().clear();

                            designCanvas.getElements().addAll(loaded);

                            designCanvas.setSelectedElement(null);

                            designCanvas.redraw();

                            stage.setScene(mainScene[0]);
                            player.play();
                        }
                )
        );
        stage.show();

        // Start session tracking after the app is fully loaded
        // Pause immediately — user is on welcome screen, not designing yet
        SessionTracker.getInstance().startSession();
        //SessionTracker.getInstance().pauseSession();
    }

    private static VBox statTile(String label, String value) {
        Label valLbl = new Label(value);
        valLbl.setStyle("-fx-text-fill: #3ddcf8; -fx-font-family: 'Segoe UI';" +
                "-fx-font-weight: bold; -fx-font-size: 16px;");
        Label keyLbl = new Label(label);
        keyLbl.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");
        VBox tile = new VBox(2, valLbl, keyLbl);
        tile.setAlignment(javafx.geometry.Pos.CENTER);
        tile.setStyle("-fx-background-color: #2b2b4a; -fx-background-radius: 10; -fx-padding: 8 14 8
14;");
        tile.setPrefWidth(80);
        return tile;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

WelcomeScreen.java

Creates the animated splash screen Scene with a looping video background, the NEW DESIGN / LOAD DESIGN / CHOOSE TEMPLATE buttons, and the template selection popup.

```
// WelcomeScreen.java
package com.framestudyo.framestudyo;

import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.*;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import javafx.scene.media.MediaView;
import javafx.stage.FileChooser;
import javafx.stage.Stage;
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import javafx.scene.control.Label;

public class WelcomeScreen {
```

```
    public static Scene create(Stage stage, Runnable onNewDesign,
java.util.function.Consumer<List<DesignElement>> onLoadDesign) {

        // --- VIDEO BACKGROUND ---
        Media media = new Media(
                WelcomeScreen.class.getResource(
                        "/com/framestudyo/framestudyo/bg1.mp4"
                ).toExternalForm()
        );

        MediaPlayer mediaPlayer = new MediaPlayer(media);
        MediaView mediaView = new MediaView(mediaPlayer);
        mediaView.setPreserveRatio(false);
        mediaView.fitWidthProperty().bind(stage.widthProperty());
        mediaView.fitHeightProperty().bind(stage.heightProperty());
        mediaPlayer.setCycleCount(MediaPlayer.INDEFINITE);
        mediaPlayer.setMute(true);
        mediaPlayer.play();

        // --- BUTTONS ---
        Button newBtn  = new Button("NEW DESIGN");
        Button loadBtn = new Button("LOAD DESIGN");

        String baseStyle =
                "-fx-font-size: 15px;" +
                        "-fx-font-family: Georgia;" +
                        "-fx-font-weight: bold;" +
                        "-fx-padding: 12 35 12 35;" +
                        "-fx-cursor: hand;" +
                        "-fx-background-radius: 8;" +
                        "-fx-border-radius: 8;" +
                        "-fx-border-width: 2.5;";

        String normalStyle = baseStyle +
                "-fx-background-color: #3e7e9f;" +
                "-fx-text-fill: white;" +
                "-fx-border-color: white;";

        String hoverStyle = baseStyle +
                "-fx-background-color: #bc4156;" +
                "-fx-text-fill: white;" +
                "-fx-border-color: white;";

        newBtn.setStyle(normalStyle);
        loadBtn.setStyle(normalStyle);

        newBtn.setOnMouseEntered(e -> newBtn.setStyle(hoverStyle));
        newBtn.setOnMouseExited(e -> newBtn.setStyle(normalStyle));
        loadBtn.setOnMouseEntered(e -> loadBtn.setStyle(hoverStyle));
        loadBtn.setOnMouseExited(e -> loadBtn.setStyle(normalStyle));

        // --- BUTTON ACTIONS ---
        newBtn.setOnAction(e -> {
            mediaPlayer.pause();
            mediaPlayer.seek(javafx.util.Duration.ZERO);
            onNewDesign.run();
        });

        loadBtn.setOnAction(e -> {
            FileChooser fileChooser = new FileChooser();
            fileChooser.setTitle("Load Design");
            fileChooser.getExtensionFilters().add(
                    new FileChooser.ExtensionFilter("FrameStudyo Files", "*.fsd")
            );
            File file = fileChooser.showOpenDialog(stage);
            if (file != null) {
                List<DesignElement> loaded = FileManager.load(file.getAbsolutePath());
                if (loaded != null) {
                    mediaPlayer.pause();
                    mediaPlayer.seek(javafx.util.Duration.ZERO);
```

```
                onLoadDesign.accept(loaded);
            }
        }
    });

    // --- TEMPLATE SECTION ---

    Button templateBtn = new Button("CHOOSE TEMPLATE");

    templateBtn.setStyle(normalStyle);

    templateBtn.setOnMouseEntered(e ->
            templateBtn.setStyle(hoverStyle));

    templateBtn.setOnMouseExited(e ->
            templateBtn.setStyle(normalStyle));

    templateBtn.setOnAction(e -> {

        Stage popup = new Stage();

        popup.initOwner(stage);

        VBox layout = new VBox(15);

        layout.setAlignment(Pos.CENTER);

        layout.setPadding(new Insets(25));

        layout.setStyle(
                "-fx-background-color: #1a1a2e;"
        );

        Label title = new Label("Select Template");

        title.setStyle(
                "-fx-text-fill: white;" +
                        "-fx-font-size: 22px;" +
                        "-fx-font-weight: bold;"
        );

        layout.getChildren().add(title);

        for (TemplateEngine.Template t :
                TemplateEngine.getTemplates()) {

            if (t == TemplateEngine.Template.BLANK)
                continue;

            Button temp = new Button(
                    t.displayName()
            );

            temp.setStyle(normalStyle);

            temp.setMaxWidth(Double.MAX_VALUE);

            temp.setOnMouseEntered(ev ->
                    temp.setStyle(hoverStyle));

            temp.setOnMouseExited(ev ->
                    temp.setStyle(normalStyle));

            temp.setOnAction(ev -> {
```

```
                    mediaPlayer.pause();
                    mediaPlayer.seek(javafx.util.Duration.ZERO);

                    List<DesignElement> els =
                            TemplateEngine.build(t);

                    popup.close();

                    onLoadDesign.accept(
                            new ArrayList<>(els)
                    );
                });

                layout.getChildren().add(temp);
            }

            Scene popScene = new Scene(layout, 350, 500);

            popup.setScene(popScene);

            popup.setTitle("Templates");

            popup.show();
        });

        // --- BUTTON ROW ---
        HBox buttonRow = new HBox(200);
        buttonRow.setAlignment(Pos.CENTER);
        buttonRow.getChildren().addAll(newBtn, loadBtn);

        // --- OVERLAY LAYOUT ---
        VBox overlay = new VBox();
        overlay.setAlignment(Pos.BOTTOM_CENTER);
        overlay.setSpacing(35);

        overlay.setPadding(new Insets(40));
        overlay.getChildren().addAll(
                templateBtn,
                buttonRow
        );
        overlay.prefWidthProperty().bind(stage.widthProperty());
        overlay.prefHeightProperty().bind(stage.heightProperty());

        // --- STACK: video + buttons, NO dark overlay ---
        StackPane root = new StackPane();
        root.getChildren().addAll(mediaView, overlay);

        Scene scene = new Scene(root);
        root.prefWidthProperty().bind(scene.widthProperty());
        root.prefHeightProperty().bind(scene.heightProperty());

        return scene;
    }
}
```

DesignCanvas.java

Wraps a JavaFX Canvas. Maintains the ordered list of DesignElement objects, exposes redraw(), add/delete/z-order helpers, and selected-element state.

```
// DesignCanvas.java
package com.framestudyo.framestudyo;


import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.paint.Color;
import java.util.ArrayList;
```

```
import java.util.List;

public class DesignCanvas {

    private Canvas canvas;
    private GraphicsContext gc;
    private List<DesignElement> elements;
    private String backgroundColor = "#2b2b2b";
    private boolean showGrid = false;

    public DesignCanvas(Canvas canvas) {
        this.canvas = canvas;
        this.gc = canvas.getGraphicsContext2D();
        this.elements = new ArrayList<>();
    }
    public boolean isShowGrid() { return showGrid; }
    public void setShowGrid(boolean showGrid) { this.showGrid = showGrid; redraw(); }
    public String getBackgroundColor() { return backgroundColor; }
    public void setBackgroundColor(String color) { this.backgroundColor = color; redraw(); }

    // Add a shape to the canvas
    public void addElement(DesignElement element) {
        elements.add(element);
        redraw();
    }
    public void updateSelectedAngle(double angle) {
        if (selectedElement != null) {
            selectedElement.setAngle(angle);
            redraw();
        }
    }

    // Redraw everything from scratch
    public void redraw() {

        // Clear the canvas
       // gc.setFill(Color.web(backgroundColor));
       // gc.fillRect(0, 0, canvas.getWidth(), canvas.getHeight());

        gc.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());

        if (!backgroundColor.equals("transparent")) {

            gc.setFill(Color.web(backgroundColor));

            gc.fillRect(0, 0,
                    canvas.getWidth(),
                    canvas.getHeight());
        }

        // Draw grid if enabled
        if (showGrid) {
            gc.setStroke(Color.web("#444444"));
            gc.setLineWidth(0.5);
            double gridSize = 20;
            for (double x = 0; x < canvas.getWidth(); x += gridSize) {
                gc.strokeLine(x, 0, x, canvas.getHeight());
            }
            for (double y = 0; y < canvas.getHeight(); y += gridSize) {
                gc.strokeLine(0, y, canvas.getWidth(), y);
            }
        }

        // Draw every shape in the list
        // This is polymorphism in action!
        for (DesignElement element : elements) {
            element.draw(gc);
        }
    }
```

```
    public List<DesignElement> getElements() { return elements; }
    private DesignElement selectedElement = null;
    public Canvas getCanvas() { return canvas; }

    public DesignElement getSelectedElement() { return selectedElement; }
    public void setSelectedElement(DesignElement element) { this.selectedElement = element; }

    public void deleteSelected() {
        System.out.println("deleteSelected called, selectedElement = " + selectedElement);
        if (selectedElement != null) {
            elements.remove(selectedElement);
            selectedElement = null;
            redraw();
        }
    }
    public void updateSelectedFillColor(String color) {
        if (selectedElement != null) {
            selectedElement.setFillColor(color);
            redraw();
        }
    }

    public void updateSelectedStrokeColor(String color) {
        if (selectedElement != null) {
            selectedElement.setStrokeColor(color);
            redraw();
        }
    }

    public void updateSelectedStrokeWidth(double width) {
        if (selectedElement != null) {
            selectedElement.setStrokeWidth(width);
            redraw();
        }
    }
    public void bringForward() {
        if (selectedElement != null) {
            int index = elements.indexOf(selectedElement);
            if (index < elements.size() - 1) {
                elements.remove(index);
                elements.add(index + 1, selectedElement);
                redraw();
            }
        }
    }

    public void sendBackward() {
        if (selectedElement != null) {
            int index = elements.indexOf(selectedElement);
            if (index > 0) {
                elements.remove(index);
                elements.add(index - 1, selectedElement);
                redraw();
            }
        }
    }

    public void bringToFront() {
        if (selectedElement != null) {
            elements.remove(selectedElement);
            elements.add(selectedElement);
            redraw();
        }
    }

    public void sendToBack() {
        if (selectedElement != null) {
            elements.remove(selectedElement);
            elements.add(0, selectedElement);
            redraw();
        }
    }
```

```
}
```

DesignElement.java

Abstract base class for all drawable elements. Declares the draw(GraphicsContext) and duplicate() abstract methods and provides concrete implementations of position, size, colour, opacity, rotation, and hit-testing.

```
// DesignElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;
import javafx.scene.paint.Color;
import java.io.Serializable;



public abstract class DesignElement implements Serializable {

    // Every shape has a position and size
    protected double x;
    protected double y;
    protected double width;
    protected double height;

    // Every shape has colors
    protected String fillColor;
    protected String strokeColor;
    protected double strokeWidth;
    protected double opacity = 1.0;
    protected double angle = 0.0;

    // Constructor
    public DesignElement(double x, double y, double width, double height) {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
        this.fillColor = "#4A90D9";
        this.strokeColor = "#000000";
        this.strokeWidth = 2.0;
    }
    public double getOpacity() { return opacity; }
    public void setOpacity(double opacity) { this.opacity = opacity; }

    public double getAngle() { return angle; }
    public void setAngle(double angle) { this.angle = angle; }

    // Every shape MUST know how to draw itself
    // This is abstract - each subclass will implement it differently
    public abstract void draw(GraphicsContext gc);
    public abstract DesignElement duplicate();
    // --- Getters and Setters ---

    public double getX() { return x; }
    public void setX(double x) { this.x = x; }

    public double getY() { return y; }
    public void setY(double y) { this.y = y; }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }

    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }

    public String getFillColor() { return fillColor; }
```

```
    public void setFillColor(String fillColor) { this.fillColor = fillColor; }

    public Color getFillColorAsColor() { return Color.web(fillColor); }

    public String getStrokeColor() { return strokeColor; }
    public void setStrokeColor(String strokeColor) { this.strokeColor = strokeColor; }

    public Color getStrokeColorAsColor() { return Color.web(strokeColor); }

    public double getStrokeWidth() { return strokeWidth; }
    public void setStrokeWidth(double strokeWidth) { this.strokeWidth = strokeWidth; }

    // Check if a point (like a mouse click) is inside this shape
    public boolean contains(double px, double py) {
        return px >= x && px <= x + width && py >= y && py <= y + height;
    }

    // Move the shape by a certain amount
    public void move(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }
}
```

RectangleElement.java

Draws a filled and stroked rectangle. Applies rotation around the element's own centre.

```
// RectangleElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;

public class RectangleElement extends DesignElement {

    // Constructor
    public RectangleElement(double x, double y, double width, double height) {
        super(x, y, width, height);
    }

    // This is where polymorphism happens!
    // RectangleElement knows how to draw itself as a rectangle
    @Override
        public void draw(GraphicsContext gc) {
            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                             // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);
            gc.setFill(getFillColorAsColor());
            gc.fillRect(x, y, width, height);
            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokeRect(x, y, width, height);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public DesignElement duplicate() {
        RectangleElement copy = new RectangleElement(x, y, width, height);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
```

```
        return copy;
    }
}
```

CircleElement.java

Draws a filled and stroked oval/circle. Applies rotation around the element's own centre.

```
// CircleElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;

public class CircleElement extends DesignElement {

    // Constructor
    public CircleElement(double x, double y, double width, double height) {
        super(x, y, width, height);
    }

    // CircleElement knows how to draw itself as an oval/circle
    @Override
        public void draw(GraphicsContext gc) {
            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                             // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);
            gc.setFill(getFillColorAsColor());
            gc.fillOval(x, y, width, height);
            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokeOval(x, y, width, height);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public DesignElement duplicate() {
        CircleElement copy = new CircleElement(x, y, width, height);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }
}
```

LineElement.java

Draws a line segment between (x,y) and (endX,endY). Overrides contains() with a point-to-segment distance test (tolerance: 5 px).

```
// LineElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;

public class LineElement extends DesignElement {

    private double endX;
    private double endY;
```

```
    public LineElement(double startX, double startY, double endX, double endY) {
        super(startX, startY, 0, 0);
        this.endX = endX;
        this.endY = endY;
    }
    @Override
        public void draw(GraphicsContext gc) {
            gc.save();
        gc.translate((x + endX) / 2, (y + endY) / 2);
        gc.rotate(angle);
        gc.translate(-((x + endX) / 2), -((y + endY) / 2));

            gc.setGlobalAlpha(opacity);
            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokeLine(x, y, endX, endY);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public boolean contains(double px, double py) {
        // Check if the point is close enough to the line (within 5 pixels)
        double tolerance = 5.0;

        // Calculate distance from point to line segment
        double dx = endX - x;
        double dy = endY - y;
        double length = Math.sqrt(dx * dx + dy * dy);

        if (length == 0) return false;

        // Project the point onto the line
        double t = ((px - x) * dx + (py - y) * dy) / (length * length);
        t = Math.max(0, Math.min(1, t));

        double closestX = x + t * dx;
        double closestY = y + t * dy;

        double distance = Math.sqrt(
                (px - closestX) * (px - closestX) +
                        (py - closestY) * (py - closestY)
        );

        return distance <= tolerance;
    }

    public double getEndX() { return endX; }
    public void setEndX(double endX) { this.endX = endX; }

    public double getEndY() { return endY; }
    public void setEndY(double endY) { this.endY = endY; }
    @Override
    public DesignElement duplicate() {
        LineElement copy = new LineElement(x, y, endX, endY);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }
}
```

TextElement.java

Renders text using a configurable font family and size. Stores text string alongside the standard DesignElement properties.

```
// TextElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;
import javafx.scene.text.Font;

public class TextElement extends DesignElement {

    private String text;
    private String fontFamily;
    private double fontSize;

    public TextElement(double x, double y, String text) {
        super(x, y, 200, 50);
        this.text = text;
        this.fontFamily = "Arial";
        this.fontSize = 24;
    }
    @Override
        public void draw(GraphicsContext gc) {
            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                             // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);
            gc.setFill(getFillColorAsColor());
        gc.setFont(new Font(fontFamily, fontSize));
            gc.fillText(text, x, y);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }



    public String getText() { return text; }
    public void setText(String text) { this.text = text; }

    public String getFontFamily() {
        return fontFamily;
    }

    public void setFontFamily(String fontFamily) {
        this.fontFamily = fontFamily;
    }

    public double getFontSize() {
        return fontSize;
    }

    public void setFontSize(double fontSize) {
        this.fontSize = fontSize;
    }
    @Override
    public DesignElement duplicate() {
        TextElement copy = new TextElement(x, y, text);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setFontFamily(fontFamily);
        copy.setFontSize(fontSize);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }
}
```

ImageElement.java

Embeds a raster image from a file path. The Image field is marked transient; loadImage() is called lazily on first draw after deserialisation.

```
// ImageElement.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;
import javafx.scene.image.Image;
import java.io.Serializable;

public class ImageElement extends DesignElement {
    private String imagePath;
    private transient Image image;

    public ImageElement(double x, double y, double width, double height, String imagePath) {
        super(x, y, width, height);
        this.imagePath = imagePath;
        this.image = new Image("file:" + imagePath);
    }

    private void loadImage() {
        if (image == null && imagePath != null)
            image = new Image("file:" + imagePath);
    }

    @Override
    public void draw(GraphicsContext gc) {
        loadImage();
        gc.save();
        gc.translate(x + width/2, y + height/2);
        gc.rotate(angle);
        gc.translate(-(x + width/2), -(y + height/2));
        gc.setGlobalAlpha(opacity);
        if (image != null) gc.drawImage(image, x, y, width, height);
        gc.setGlobalAlpha(1.0);
        gc.restore();
    }

    @Override
    public DesignElement duplicate() {
        ImageElement copy = new ImageElement(x, y, width, height, imagePath);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }

    public String getImagePath() { return imagePath; }
}
```

Shapes.java

Contains three additional shape classes: TriangleElement (polygon with 3 vertices), StarElement (10-vertex polygon with outer/inner radii), and ArrowElement (7-vertex polygon).

```
// Shapes.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.GraphicsContext;


class TriangleElement extends DesignElement {
```

```
    public TriangleElement(double x, double y, double width, double height) {
        super(x, y, width, height);
    }

        @Override
        public void draw(GraphicsContext gc) {
            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                            // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);
            double[] xPoints = { x + width / 2, x, x + width };
            double[] yPoints = { y, y + height, y + height };

            gc.setFill(getFillColorAsColor());
            gc.fillPolygon(xPoints, yPoints, 3);

            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokePolygon(xPoints, yPoints, 3);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public DesignElement duplicate() {
        TriangleElement copy = new TriangleElement(x, y, width, height);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }
}


class StarElement extends DesignElement {

    public StarElement(double x, double y, double width, double height) {
        super(x, y, width, height);
    }

    @Override
    public void draw(GraphicsContext gc) {
        gc.setGlobalAlpha(opacity);
            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                            // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);
            double cx = x + width / 2;
            double cy = y + height / 2;
            double outerR = Math.min(width, height) / 2;
            double innerR = outerR * 0.4;
            int points = 5;

            double[] xPoints = new double[points * 2];
            double[] yPoints = new double[points * 2];

            for (int i = 0; i < points * 2; i++) {
                double angle = Math.PI / points * i - Math.PI / 2;
                double r = (i % 2 == 0) ? outerR : innerR;
                xPoints[i] = cx + r * Math.cos(angle);
                yPoints[i] = cy + r * Math.sin(angle);
```

```
            }

            gc.setFill(getFillColorAsColor());
            gc.fillPolygon(xPoints, yPoints, points * 2);

            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokePolygon(xPoints, yPoints, points * 2);
            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public DesignElement duplicate() {
        StarElement copy = new StarElement(x, y, width, height);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);
        return copy;
    }
}


class ArrowElement extends DesignElement {

    public ArrowElement(double x, double y, double width, double height) {
        super(x, y, width, height);
    }

    @Override
    public void draw(GraphicsContext gc) {

            gc.save();
            gc.translate(x + width / 2, y + height / 2); // move to center
            gc.rotate(angle);                                  // rotate
            gc.translate(-(x + width / 2), -(y + height / 2)); // move back

            gc.setGlobalAlpha(opacity);

            gc.setGlobalAlpha(opacity);
            double shaftH = height * 0.4;
            double shaftY = y + height * 0.3;
            double headW  = width * 0.4;

            double[] xPoints = {
                    x,                  // left of shaft
                    x + width - headW,  // where shaft meets head
                    x + width - headW,  // top of arrowhead
                    x + width,          // tip
                    x + width - headW,  // bottom of arrowhead
                    x + width - headW,  // where shaft meets head (bottom)
                    x                   // back to start
            };

            double[] yPoints = {
                    shaftY,             // top left
                    shaftY,             // top right of shaft
                    y,                  // top of arrowhead
                    y + height / 2,     // tip
                    y + height,         // bottom of arrowhead
                    shaftY + shaftH,    // bottom right of shaft
                    shaftY + shaftH     // bottom left
            };

            gc.setFill(getFillColorAsColor());
            gc.fillPolygon(xPoints, yPoints, 7);
```

```
            gc.setStroke(getStrokeColorAsColor());
            gc.setLineWidth(strokeWidth);
            gc.strokePolygon(xPoints, yPoints, 7);

            gc.setGlobalAlpha(1.0);

            gc.restore();
        }

    @Override
    public DesignElement duplicate() {
        ArrowElement copy = new ArrowElement(x, y, width, height);
        copy.setFillColor(fillColor);
        copy.setStrokeColor(strokeColor);
        copy.setStrokeWidth(strokeWidth);
        copy.setOpacity(opacity);
        copy.setAngle(angle);

        return copy;
    }
}
```

Commands.java

Defines the Command interface and all concrete command classes (AddShape, DeleteShape, MoveShape, ClearCanvas) plus CommandManager with the undo/redo stacks.

```
// Commands.java
package com.framestudyo.framestudyo;

import java.util.Stack;

// ── Command Interface ──────────────────────────────────────────
interface Command {
    void execute();
    void undo();
}

// ── Add Shape Command ──────────────────────────────────────────
class AddShapeCommand implements Command {

    private DesignCanvas designCanvas;
    private DesignElement element;

    public AddShapeCommand(DesignCanvas designCanvas, DesignElement element) {
        this.designCanvas = designCanvas;
        this.element = element;
    }

    @Override
    public void execute() {
        designCanvas.getElements().add(element);
        designCanvas.redraw();
    }

    @Override
    public void undo() {
        designCanvas.getElements().remove(element);
        designCanvas.redraw();
    }
}

// ── Delete Shape Command ───────────────────────────────────────
class DeleteShapeCommand implements Command {

    private DesignCanvas designCanvas;
```

```
    private DesignElement element;

    public DeleteShapeCommand(DesignCanvas designCanvas, DesignElement element) {
        this.designCanvas = designCanvas;
        this.element = element;
    }

    @Override
    public void execute() {
        designCanvas.getElements().remove(element);
        designCanvas.redraw();
    }

    @Override
    public void undo() {
        designCanvas.getElements().add(element);
        designCanvas.redraw();
    }
}

// ── Move Shape Command ──────────────────────────────────────────
class MoveShapeCommand implements Command {

    private DesignElement element;
    private double oldX, oldY;
    private double newX, newY;

    public MoveShapeCommand(DesignElement element, double oldX, double oldY, double newX, double newY) {
        this.element = element;
        this.oldX = oldX;
        this.oldY = oldY;
        this.newX = newX;
        this.newY = newY;
    }

    @Override
    public void execute() {
        element.setX(newX);
        element.setY(newY);
    }

    @Override
    public void undo() {
        element.setX(oldX);
        element.setY(oldY);
    }
}

// ── Clear Canvas Command ────────────────────────────────────────
class ClearCanvasCommand implements Command {

    private DesignCanvas designCanvas;
    private java.util.List<DesignElement> backup;

    public ClearCanvasCommand(DesignCanvas designCanvas) {
        this.designCanvas = designCanvas;
        this.backup = new java.util.ArrayList<>(designCanvas.getElements());
    }

    @Override
    public void execute() {
        designCanvas.getElements().clear();
        designCanvas.redraw();
    }

    @Override
    public void undo() {
        designCanvas.getElements().clear();
        designCanvas.getElements().addAll(backup);
        designCanvas.redraw();
```

```
    }
}

// ── Command Manager (the Undo/Redo stack) ──────────────────────────────
class CommandManager {

    private Stack<Command> undoStack = new Stack<>();
    private Stack<Command> redoStack = new Stack<>();

    public void executeCommand(Command command) {
        command.execute();
        undoStack.push(command);
        redoStack.clear();
    }

    public void undo() {
        if (!undoStack.isEmpty()) {
            Command command = undoStack.pop();
            command.undo();
            redoStack.push(command);
        }
    }

    public void redo() {
        if (!redoStack.isEmpty()) {
            Command command = redoStack.pop();
            command.execute();
            undoStack.push(command);
        }
    }

    public boolean canUndo() { return !undoStack.isEmpty(); }
    public boolean canRedo() { return !redoStack.isEmpty(); }
}
```

AlignmentEngine.java

Snap-to-align engine used during element drag. Tests five edge-pairs per element (left, right, top, bottom, centre) within an 8-pixel threshold and records guide lines for rendering.

```
// AlignmentEngine.java
package com.framestudyo.framestudyo;

import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.paint.Color;

import java.util.ArrayList;
import java.util.List;

/**
 * AlignmentEngine

 * Provides smart snap-to-align behaviour during element drag.
 * Detects when the dragged element's edges or center align with
 * any other element on the canvas and:
 *   1. Snaps the position to the nearest alignment guide.
 *   2. Draws cyan guide lines on the canvas overlay.
 *
 * Integration in HelloApplication (inside canvas.setOnMouseDragged,
 * inside the "select" tool / non-resizing drag branch):
 *
 *   DesignElement el = designCanvas.getSelectedElement();
 *   double newX = endX - dragOffsetX;
 *   double newY = endY - dragOffsetY;
 *   double[] snapped = alignmentEngine.snap(el, newX, newY, designCanvas.getElements());
 *   el.setX(snapped[0]);
 *   el.setY(snapped[1]);
 *   designCanvas.redraw();
```

```
 *   alignmentEngine.drawGuides(canvas, designCanvas);   // draw snap lines on top
 *   drawSelection(canvas, designCanvas);
 *
 * And on mouse released / press:
 *   alignmentEngine.clearGuides(canvas, designCanvas);
 *
 * Instantiate once in HelloApplication:
 *   AlignmentEngine alignmentEngine = new AlignmentEngine();
 */
public class AlignmentEngine {

    private static final double SNAP_THRESHOLD = 8.0;   // pixels within which snap activates
    private static final Color  GUIDE_COLOR    = Color.web("#3ddcf8");
    private static final double GUIDE_WIDTH    = 1.0;

    // Guides to draw this frame
    private final List<double[]> hGuides = new ArrayList<>(); // [y, x1, x2]
    private final List<double[]> vGuides = new ArrayList<>(); // [x, y1, y2]

    // ── Snap logic ──────────────────────────────────────────────────────

    /**
     * Given a desired (newX, newY) for the dragged element, checks all other
     * elements for alignment opportunities and returns a snapped [x, y].
     * Also records which guide lines to draw.
     */
    public double[] snap(DesignElement dragged, double newX, double newY,
                         List<DesignElement> allElements) {
        hGuides.clear();
        vGuides.clear();

        double snappedX = newX;
        double snappedY = newY;

        // Candidate edges of the dragged element at proposed position
        double dL = newX;
        double dR = newX + dragged.getWidth();
        double dCX = newX + dragged.getWidth() / 2.0;
        double dT = newY;
        double dB = newY + dragged.getHeight();
        double dCY = newY + dragged.getHeight() / 2.0;

        double bestDX = SNAP_THRESHOLD + 1;
        double bestDY = SNAP_THRESHOLD + 1;

        for (DesignElement other : allElements) {
            if (other == dragged) continue;

            double oL  = other.getX();
            double oR  = other.getX() + other.getWidth();
            double oCX = other.getX() + other.getWidth() / 2.0;
            double oT  = other.getY();
            double oB  = other.getY() + other.getHeight();
            double oCY = other.getY() + other.getHeight() / 2.0;

            // ── Horizontal snap (X axis) ──────────────────────────────
            // Left-to-left
            double d = Math.abs(dL - oL);
            if (d < bestDX) { bestDX = d; snappedX = oL; }
            // Right-to-right
            d = Math.abs(dR - oR);
            if (d < bestDX) { bestDX = d; snappedX = oR - dragged.getWidth(); }
            // Left-to-right
            d = Math.abs(dL - oR);
            if (d < bestDX) { bestDX = d; snappedX = oR; }
            // Right-to-left
            d = Math.abs(dR - oL);
            if (d < bestDX) { bestDX = d; snappedX = oL - dragged.getWidth(); }
            // Center-to-center X
            d = Math.abs(dCX - oCX);
            if (d < bestDX) { bestDX = d; snappedX = oCX - dragged.getWidth() / 2.0; }
```

```
            // ── Vertical snap (Y axis) ──────────────────────────────
            // Top-to-top
            d = Math.abs(dT - oT);
            if (d < bestDY) { bestDY = d; snappedY = oT; }
            // Bottom-to-bottom
            d = Math.abs(dB - oB);
            if (d < bestDY) { bestDY = d; snappedY = oB - dragged.getHeight(); }
            // Top-to-bottom
            d = Math.abs(dT - oB);
            if (d < bestDY) { bestDY = d; snappedY = oB; }
            // Bottom-to-top
            d = Math.abs(dB - oT);
            if (d < bestDY) { bestDY = d; snappedY = oT - dragged.getHeight(); }
            // Center-to-center Y
            d = Math.abs(dCY - oCY);
            if (d < bestDY) { bestDY = d; snappedY = oCY - dragged.getHeight() / 2.0; }
        }

        // Build guide lines only when snap actually triggered
        buildGuideLines(dragged, snappedX, snappedY, allElements, bestDX, bestDY);

        return new double[]{snappedX, snappedY};
    }

    // ── Guide drawing ──────────────────────────────────────────────────

    /**
     * Draws the recorded snap guide lines onto the canvas overlay.
     * Call AFTER designCanvas.redraw() and BEFORE drawSelection().
     */
    public void drawGuides(Canvas canvas, DesignCanvas designCanvas) {
        if (hGuides.isEmpty() && vGuides.isEmpty()) return;

        GraphicsContext gc = canvas.getGraphicsContext2D();
        gc.setStroke(GUIDE_COLOR);
        gc.setLineWidth(GUIDE_WIDTH);
        gc.setLineDashes(4, 4);

        for (double[] g : hGuides) { // [y, x1, x2]
            gc.strokeLine(g[1], g[0], g[2], g[0]);
        }
        for (double[] g : vGuides) { // [x, y1, y2]
            gc.strokeLine(g[0], g[1], g[0], g[2]);
        }

        gc.setLineDashes(0);
    }

    /** Clears guides — call on mouse press and mouse released. */
    public void clearGuides(Canvas canvas, DesignCanvas designCanvas) {
        hGuides.clear();
        vGuides.clear();
        designCanvas.redraw();
    }

    // ── Internal helpers ──────────────────────────────────────────────

    private void buildGuideLines(DesignElement dragged, double snappedX, double snappedY,
                                 List<DesignElement> allElements, double bestDX, double bestDY) {
        double cW = 860, cH = 600; // canvas size — adjust if yours differs

        if (bestDX <= SNAP_THRESHOLD) {
            // Vertical guide line at snapped X edge
            vGuides.add(new double[]{snappedX, 0, cH});
            vGuides.add(new double[]{snappedX + dragged.getWidth(), 0, cH});
        }
        if (bestDY <= SNAP_THRESHOLD) {
            // Horizontal guide line at snapped Y edge
            hGuides.add(new double[]{snappedY, 0, cW});
```

```
            hGuides.add(new double[]{snappedY + dragged.getHeight(), 0, cW});
        }
    }
}
```

ShapeRecognizer.java

Records freehand stroke points and classifies them on mouse release. Uses bounding box aspect ratio, linearity deviation, closure, corner count, and circularity to return the best-matching DesignElement subtype.

```
// ShapeRecognizer.java
package com.framestudyo.framestudyo;

import java.util.ArrayList;
import java.util.List;



public class ShapeRecognizer {

    private final List<double[]> points = new ArrayList<>(); // [x, y] pairs

    // ── Stroke recording ──────────────────────────────────────────

    public void startStroke(double x, double y) {
        points.clear();
        points.add(new double[]{x, y});
    }

    public void addPoint(double x, double y) {
        // Sub-sample: only add if moved >3px to avoid noise
        if (!points.isEmpty()) {
            double[] last = points.get(points.size() - 1);
            double dx = x - last[0], dy = y - last[1];
            if (dx*dx + dy*dy < 9) return;
        }
        points.add(new double[]{x, y});
    }

    /** Draws the current freehand trail using the provided GraphicsContext. */
    public void drawStroke(javafx.scene.canvas.GraphicsContext gc) {
        if (points.size() < 2) return;
        gc.setStroke(javafx.scene.paint.Color.web("#3ddcf8"));
        gc.setLineWidth(1.5);
        gc.setLineDashes(0);
        gc.beginPath();
        gc.moveTo(points.get(0)[0], points.get(0)[1]);
        for (int i = 1; i < points.size(); i++) {
            gc.lineTo(points.get(i)[0], points.get(i)[1]);
        }
        gc.stroke();
    }

    // ── Recognition ───────────────────────────────────────────────

    /**
     * Analyzes recorded points and returns the best-matching DesignElement,
     * or null if the stroke is too short to recognize.
     */
    public DesignElement recognize(String fillColor, String strokeColor, double strokeWidth) {
        if (points.size() < 4) return null;

        double[] bbox = boundingBox();
        double bX = bbox[0], bY = bbox[1], bW = bbox[2], bH = bbox[3];

        // Need a meaningful stroke
        if (bW < 10 && bH < 10) return null;
```

```
        String shape = classify(bW, bH);

        DesignElement el;
        switch (shape) {
            case "line":
                double[] first = points.get(0);
                double[] last  = points.get(points.size() - 1);
                el = new LineElement(first[0], first[1], last[0], last[1]);
                break;
            case "circle":
                el = new CircleElement(bX, bY, bW, bH);
                el.setFillColor(fillColor);
                break;
            case "triangle":
                el = new TriangleElement(bX, bY, bW, bH);
                el.setFillColor(fillColor);
                break;
            case "star":
                el = new StarElement(bX, bY, bW, bH);
                el.setFillColor(fillColor);
                break;
            default: // rectangle
                el = new RectangleElement(bX, bY, bW, bH);
                el.setFillColor(fillColor);
                break;
        }
        el.setStrokeColor(strokeColor);
        el.setStrokeWidth(strokeWidth);
        return el;
    }

    /** Returns the recognized shape name as a human-readable label. */
    public String lastRecognizedLabel() {
        if (points.size() < 4) return "";
        double[] bbox = boundingBox();
        return classify(bbox[2], bbox[3]);
    }

    // ── Internal classification ──────────────────────────────────

    /**
     * Classifies the stroke based on:
     * - Linearity  → line
     * - Closure    → closed shape
     * - Aspect ratio + corner count → rectangle vs circle vs triangle
     */
    private String classify(double bW, double bH) {
        // 1. Line check: very narrow bounding box in one direction
        double aspectRatio = (bW + bH == 0) ? 1 : Math.max(bW, bH) / Math.max(1, Math.min(bW, bH));
        if (aspectRatio > 4.0 && isLinear()) return "line";

        // 2. Closure check
        boolean closed = isClosed();

        // 3. Corner detection
        int corners = countCorners();

        if (!closed) {
            // Treat open strokes with narrow bbox as lines
            if (aspectRatio > 2.5) return "line";
            return "rectangle"; // fallback
        }

        // 4. Circularity: how close to a circle?
        double circularity = computeCircularity(bW, bH);
        if (circularity > 0.90 && corners < 4)
            return "circle";

        // 5. Corner count
        if (corners >= 2 && corners <= 5)
```

```
            return "triangle";

        //6.Detect Star
        if (corners >= 8)
            return "star";

        // 7. Default → rectangle
        return "rectangle";



    }

    /** True if start and end points are within 15% of the bounding box diagonal. */
    private boolean isClosed() {
        if (points.size() < 6) return false;
        double[] first = points.get(0);
        double[] last  = points.get(points.size() - 1);
        double[] bbox  = boundingBox();
        double diag    = Math.sqrt(bbox[2]*bbox[2] + bbox[3]*bbox[3]);
        double dist    = Math.sqrt(Math.pow(last[0]-first[0],2) + Math.pow(last[1]-first[1],2));
        return dist < diag * 0.25;
    }

    /** True if points form an approximately straight line (low deviation from linear fit). */
    private boolean isLinear() {
        if (points.size() < 3) return true;
        double[] first = points.get(0);
        double[] last  = points.get(points.size() - 1);
        double dx = last[0] - first[0], dy = last[1] - first[1];
        double len = Math.sqrt(dx*dx + dy*dy);
        if (len == 0) return true;
        double maxDev = 0;
        for (double[] p : points) {
            // Distance from point to the line first→last
            double dev = Math.abs(dy*p[0] - dx*p[1] + last[0]*first[1] - last[1]*first[0]) / len;
            if (dev > maxDev) maxDev = dev;
        }
        return maxDev < 12; // within 12px of the line
    }

    /**
     * Counts approximate corners by detecting direction changes > 40°.
     * Returns 0-6.
     */
    private int countCorners() {
        if (points.size() < 5) return 0;
        int corners = 0;
        int step = Math.max(1, points.size() / 20);
        for (int i = step; i < points.size() - step; i += step) {
            double[] prev = points.get(i - step);
            double[] curr = points.get(i);
            double[] next = points.get(Math.min(i + step, points.size()-1));
            double ax = curr[0]-prev[0], ay = curr[1]-prev[1];
            double bx = next[0]-curr[0], by = next[1]-curr[1];
            double dot   = ax*bx + ay*by;
            double cross = ax*by - ay*bx;
            double angle = Math.toDegrees(Math.atan2(Math.abs(cross), dot));
            if (angle > 40) corners++;
        }
        return corners;
    }

    /**
     * Measures how circular the stroke is.
     * Compares perimeter to what a circle of the same bounding-box size would have.
     * Returns 0 (not circular) to 1 (perfect circle).
     */
    private double computeCircularity(double bW, double bH) {
        if (bW <= 0 || bH <= 0) return 0;
        double strokeLen = strokeLength();
        if (strokeLen == 0) return 0;
        double r = (bW + bH) / 4.0;
```

```
        double idealPerimeter = 2 * Math.PI * r;
        return Math.min(1.0, idealPerimeter / strokeLen);
    }

    private double strokeLength() {
        double len = 0;
        for (int i = 1; i < points.size(); i++) {
            double dx = points.get(i)[0] - points.get(i-1)[0];
            double dy = points.get(i)[1] - points.get(i-1)[1];
            len += Math.sqrt(dx*dx + dy*dy);
        }
        return len;
    }

    /** Returns [minX, minY, width, height] of the stroke bounding box. */
    private double[] boundingBox() {
        double minX = Double.MAX_VALUE, minY = Double.MAX_VALUE;
        double maxX = Double.MIN_VALUE, maxY = Double.MIN_VALUE;
        for (double[] p : points) {
            if (p[0] < minX) minX = p[0];
            if (p[1] < minY) minY = p[1];
            if (p[0] > maxX) maxX = p[0];
            if (p[1] > maxY) maxY = p[1];
        }
        return new double[]{minX, minY, maxX-minX, maxY-minY};
    }
}
```

SmartColorAdvisor.java

Analyses canvas fill colours and generates complementary, triadic, and tint suggestions using HSL arithmetic. Renders a floating swatch panel; clicking a swatch copies the hex code.

```
// SmartColorAdvisor.java
package com.framestudyo.framestudyo;

import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Tooltip;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Modality;
import javafx.stage.Stage;
import javafx.stage.StageStyle;

import java.util.*;

/**
 * SmartColorAdvisor
 *
 * Analyzes all fill colors currently on the DesignCanvas and suggests
 * a complementary color palette using HSL color theory.
 *
 * Usage in HelloApplication:
 *   Button paletteBtn = styledBtn("🎨 Palette", "#6a3d8f", "#9b59b6");
 *   paletteBtn.setOnAction(e -> SmartColorAdvisor.show(stage, designCanvas));
 */
public class SmartColorAdvisor {

    // ── Public entry point ──────────────────────────────────────────

    public static void show(Stage owner, DesignCanvas designCanvas) {
        List<String> canvasColors = extractColors(designCanvas);
        List<String> suggested   = generateSuggestions(canvasColors);

        Stage popup = new Stage();
```

```
        popup.initOwner(owner);
        popup.initModality(Modality.NONE);
        popup.initStyle(StageStyle.UNDECORATED);
        popup.setTitle("Smart Palette");

        VBox root = new VBox(14);
        root.setPadding(new Insets(18));
        root.setStyle(
                "-fx-background-color: #1e1235;" +
                        "-fx-border-color: #3ddcf8;" +
                        "-fx-border-width: 1.5;" +
                        "-fx-border-radius: 12;" +
                        "-fx-background-radius: 12;"
        );
        root.setPrefWidth(280);

        // Title
        Label title = new Label("✦ Smart Color Palette");
        title.setStyle("-fx-text-fill: #3ddcf8; -fx-font-family: 'Segoe UI'; " +
                "-fx-font-weight: bold; -fx-font-size: 14px;");

        // Canvas colors section
        Label canvasLbl = new Label("Colors on canvas:");
        canvasLbl.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; " +
                "-fx-font-size: 11px; -fx-font-weight: bold;");
        HBox canvasRow = buildColorRow(canvasColors.isEmpty()
                ? List.of("#ffffff") : canvasColors, "Canvas color");

        // Suggested colors section
        Label suggestLbl = new Label("Suggested complements:");
        suggestLbl.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; " +
                "-fx-font-size: 11px; -fx-font-weight: bold;");
        HBox suggestRow = buildColorRow(suggested, "Suggested color");

        // Harmony type labels
        Label harmonyLbl = new Label("Harmony: Complementary + Triadic split");
        harmonyLbl.setStyle("-fx-text-fill: #888; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");

        // Close button
        javafx.scene.control.Button closeBtn = new javafx.scene.control.Button("Close");
        closeBtn.setStyle(
                "-fx-background-color: #3e7e9f; -fx-text-fill: white;" +
                        "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                        "-fx-background-radius: 20; -fx-padding: 4 14 4 14; -fx-cursor: hand;"
        );
        closeBtn.setOnAction(e -> popup.close());
        HBox btnRow = new HBox(closeBtn);
        btnRow.setAlignment(Pos.CENTER_RIGHT);

        root.getChildren().addAll(title, canvasLbl, canvasRow,
                new javafx.scene.control.Separator(),
                suggestLbl, suggestRow, harmonyLbl, btnRow);

        popup.setScene(new Scene(root));
        popup.show();

        // Position near owner
        popup.setX(owner.getX() + owner.getWidth() - 310);
        popup.setY(owner.getY() + 80);
    }

    // ── Color extraction ──────────────────────────────────────────

    private static List<String> extractColors(DesignCanvas dc) {
        Set<String> seen = new LinkedHashSet<>();
        for (DesignElement el : dc.getElements()) {
            if (el.getFillColor() != null && !el.getFillColor().isEmpty()
                    && !el.getFillColor().equalsIgnoreCase("transparent")) {
                seen.add(el.getFillColor().toUpperCase());
            }
```

```
        }
        List<String> list = new ArrayList<>(seen);
        // Cap display to 6
        return list.size() > 6 ? list.subList(0, 6) : list;
    }

    // ── Suggestion engine ──────────────────────────────────────────

    private static List<String> generateSuggestions(List<String> hexColors) {
        List<String> suggestions = new ArrayList<>();

        if (hexColors.isEmpty()) {
            // Default pleasant palette when canvas is empty
            suggestions.addAll(List.of("#FF6B9D", "#C44DFF", "#4DAFFF", "#43E8A0", "#FFD166"));
            return suggestions;
        }

        Set<String> used = new HashSet<>(hexColors);

        for (String hex : hexColors) {
            double[] hsl = hexToHsl(hex);

            // Complementary (180° flip)
            String comp = hslToHex((hsl[0] + 180) % 360, hsl[1], hsl[2]);
            if (!used.contains(comp.toUpperCase())) {
                suggestions.add(comp);
                used.add(comp.toUpperCase());
            }

            // Triadic split +120°
            String tri1 = hslToHex((hsl[0] + 120) % 360, hsl[1], hsl[2]);
            if (!used.contains(tri1.toUpperCase())) {
                suggestions.add(tri1);
                used.add(tri1.toUpperCase());
            }

            // Triadic split +240°
            String tri2 = hslToHex((hsl[0] + 240) % 360, hsl[1], hsl[2]);
            if (!used.contains(tri2.toUpperCase())) {
                suggestions.add(tri2);
                used.add(tri2.toUpperCase());
            }

            // Slightly lighter tint
            String tint = hslToHex(hsl[0], Math.max(0, hsl[1] - 15),
                    Math.min(95, hsl[2] + 20));
            if (!used.contains(tint.toUpperCase())) {
                suggestions.add(tint);
                used.add(tint.toUpperCase());
            }

            if (suggestions.size() >= 6) break;
        }

        return suggestions.size() > 6 ? suggestions.subList(0, 6) : suggestions;
    }

    // ── UI helpers ─────────────────────────────────────────────────

    private static HBox buildColorRow(List<String> hexColors, String tooltipBase) {
        HBox row = new HBox(8);
        row.setAlignment(Pos.CENTER_LEFT);
        for (String hex : hexColors) {
            Rectangle swatch = new Rectangle(36, 36);
            swatch.setArcWidth(8);
            swatch.setArcHeight(8);
            try {
                swatch.setFill(Color.web(hex));
            } catch (Exception ex) {
                swatch.setFill(Color.GRAY);
```

```
            }
            swatch.setStroke(Color.web("#3ddcf8"));
            swatch.setStrokeWidth(1);
            Tooltip.install(swatch, new Tooltip(tooltipBase + ": " + hex));

            // Click to copy hex to clipboard
            swatch.setOnMouseClicked(e -> {
                javafx.scene.input.Clipboard cb =
                        javafx.scene.input.Clipboard.getSystemClipboard();
                javafx.scene.input.ClipboardContent content =
                        new javafx.scene.input.ClipboardContent();
                content.putString(hex);
                cb.setContent(content);
            });
            swatch.setCursor(javafx.scene.Cursor.HAND);
            row.getChildren().add(swatch);
        }
        return row;
    }

    // ── Color math ──────────────────────────────────────────────────────

    /** Converts #RRGGBB → [hue°, saturation%, lightness%] */
    private static double[] hexToHsl(String hex) {
        try {
            Color c = Color.web(hex);
            double r = c.getRed(), g = c.getGreen(), b = c.getBlue();
            double max = Math.max(r, Math.max(g, b));
            double min = Math.min(r, Math.min(g, b));
            double l = (max + min) / 2.0;
            double s = 0, h = 0;
            if (max != min) {
                double d = max - min;
                s = l > 0.5 ? d / (2 - max - min) : d / (max + min);
                if (max == r)      h = (g - b) / d + (g < b ? 6 : 0);
                else if (max == g) h = (b - r) / d + 2;
                else               h = (r - g) / d + 4;
                h *= 60;
            }
            return new double[]{h, s * 100, l * 100};
        } catch (Exception e) {
            return new double[]{0, 0, 50};
        }
    }

    /** Converts [hue°, saturation%, lightness%] → #RRGGBB */
    private static String hslToHex(double h, double s, double l) {
        double sn = s / 100.0, ln = l / 100.0;
        double c = (1 - Math.abs(2 * ln - 1)) * sn;
        double x = c * (1 - Math.abs((h / 60) % 2 - 1));
        double m = ln - c / 2;
        double r = 0, g = 0, b = 0;
        int sector = (int)(h / 60) % 6;
        switch (sector) {
            case 0: r=c; g=x; break;
            case 1: r=x; g=c; break;
            case 2: g=c; b=x; break;
            case 3: g=x; b=c; break;
            case 4: r=x; b=c; break;
            case 5: r=c; b=x; break;
        }
        return String.format("#%02X%02X%02X",
                (int)((r+m)*255), (int)((g+m)*255), (int)((b+m)*255));
    }
}
```

DesignAnalyzer.java

Computes balance (visual centre-of-mass), contrast (luminance variance), density (area coverage), and complexity (element count) scores and displays them in a styled modal popup.

```
// DesignAnalyzer.java
package com.framestudyo.framestudyo;

import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Modality;
import javafx.stage.Stage;
import javafx.stage.StageStyle;

import java.util.List;

/**
 * DesignAnalyzer
 *
 * Analyzes the current canvas and gives a scored design report across:
 *   - Balance     : how centered is the visual weight?
 *   - Contrast    : how varied are the colors?
 *   - Density     : how much of the canvas is used?
 *   - Complexity  : is there too much or too little going on?
 *
 * Usage in HelloApplication:
 *   Button analyzeBtn = styledBtn("📊 Analyze", "#2b588b", "#3e7e9f");
 *   analyzeBtn.setOnAction(e ->
 *       DesignAnalyzer.show(stage, designCanvas, 860, 600));
 */
public class DesignAnalyzer {

    private static final double CANVAS_W = 860;
    private static final double CANVAS_H = 600;

    // ── Public entry point ──────────────────────────────────────────

    public static void show(Stage owner, DesignCanvas designCanvas,
                            double canvasW, double canvasH) {

        List<DesignElement> elements = designCanvas.getElements();

        // Compute scores (0–100)
        int balanceScore   = scoreBalance(elements, canvasW, canvasH);
        int contrastScore  = scoreContrast(elements);
        int densityScore   = scoreDensity(elements, canvasW, canvasH);
        int complexityScore = scoreComplexity(elements);
        int overallScore   = (balanceScore + contrastScore + densityScore + complexityScore) / 4;

        // Build the popup
        Stage popup = new Stage();
        popup.initOwner(owner);
        popup.initModality(Modality.APPLICATION_MODAL);
        popup.initStyle(StageStyle.UNDECORATED);
        popup.setTitle("Design Analysis");

        VBox root = new VBox(14);
        root.setPadding(new Insets(20));
        root.setPrefWidth(320);
        root.setStyle(
                "-fx-background-color: #1e1235;" +
                        "-fx-border-color: #3ddcf8;" +
                        "-fx-border-width: 1.5;" +
                        "-fx-border-radius: 14;" +
                        "-fx-background-radius: 14;"
        );

        Label title = new Label("✦ Design Score");
        title.setStyle("-fx-text-fill: #3ddcf8; -fx-font-family: 'Segoe UI'; " +
                "-fx-font-weight: bold; -fx-font-size: 15px;");
```

```
        // Overall score ring (text-based)
        Label overallLbl = new Label(overallScore + " / 100");
        overallLbl.setStyle(
                "-fx-text-fill: " + scoreColor(overallScore) + ";" +
                        "-fx-font-family: 'Segoe UI'; -fx-font-weight: bold; -fx-font-size: 28px;"
        );
        Label overallTag = new Label(overallTag(overallScore));
        overallTag.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; -fx-font-size:
12px;");

        VBox overallBox = new VBox(2, overallLbl, overallTag);
        overallBox.setAlignment(Pos.CENTER);
        overallBox.setStyle(
                "-fx-background-color: rgba(61,220,248,0.08);" +
                        "-fx-background-radius: 10;" +
                        "-fx-padding: 10;"
        );

        // Individual score rows
        VBox scores = new VBox(8);
        scores.getChildren().addAll(
                scoreRow("⚖ Balance",    balanceScore,   feedbackBalance(balanceScore)),
                scoreRow("🎨 Contrast",   contrastScore,  feedbackContrast(contrastScore)),
                scoreRow("📐 Density",    densityScore,   feedbackDensity(densityScore)),
                scoreRow("🔢 Complexity", complexityScore, feedbackComplexity(complexityScore,
elements.size()))
        );

        // Element count summary
        Label summary = new Label(
                "Elements: " + elements.size() +
                        "  |  Canvas: " + (int)canvasW + "×" + (int)canvasH
        );
        summary.setStyle("-fx-text-fill: #888; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");

        javafx.scene.control.Button closeBtn = new javafx.scene.control.Button("Close");
        closeBtn.setStyle(
                "-fx-background-color: #3e7e9f; -fx-text-fill: white;" +
                        "-fx-font-family: 'Segoe UI'; -fx-font-size: 11px;" +
                        "-fx-background-radius: 20; -fx-padding: 5 16 5 16; -fx-cursor: hand;"
        );
        closeBtn.setOnAction(e -> popup.close());
        HBox btnRow = new HBox(closeBtn);
        btnRow.setAlignment(Pos.CENTER_RIGHT);

        root.getChildren().addAll(
                title, overallBox,
                new javafx.scene.control.Separator(),
                scores,
                new javafx.scene.control.Separator(),
                summary, btnRow
        );

        popup.setScene(new Scene(root));
        popup.show();
        popup.setX(owner.getX() + (owner.getWidth()  - popup.getWidth())  / 2);
        popup.setY(owner.getY() + (owner.getHeight() - popup.getHeight()) / 2);
    }

    // ── Score computations ──────────────────────────────────────

    /**
     * Balance: distance of visual center-of-mass from canvas center.
     * Perfect center = 100, far off = 0.
     */
    private static int scoreBalance(List<DesignElement> elements, double cW, double cH) {
        if (elements.isEmpty()) return 50;
        double cx = 0, cy = 0;
        for (DesignElement el : elements) {
            cx += el.getX() + el.getWidth()  / 2.0;
```

```
            cy += el.getY() + el.getHeight() / 2.0;
        }
        cx /= elements.size();
        cy /= elements.size();
        double maxDist = Math.sqrt(cW*cW + cH*cH) / 2.0;
        double dist    = Math.sqrt(Math.pow(cx - cW/2, 2) + Math.pow(cy - cH/2, 2));
        return (int) Math.max(0, Math.min(100, 100 - (dist / maxDist) * 100));
    }

    /**
     * Contrast: color variance across elements.
     * High variance = high contrast = good.
     */
    private static int scoreContrast(List<DesignElement> elements) {
        if (elements.size() < 2) return 40;
        double[] luminances = elements.stream()
                .filter(el -> el.getFillColor() != null && !el.getFillColor().isEmpty())
                .mapToDouble(el -> luminance(el.getFillColor()))
                .toArray();
        if (luminances.length < 2) return 40;
        double mean = 0;
        for (double l : luminances) mean += l;
        mean /= luminances.length;
        double variance = 0;
        for (double l : luminances) variance += Math.pow(l - mean, 2);
        variance /= luminances.length;
        // Map variance (0–0.25 typical) to 0–100
        return (int) Math.min(100, (variance / 0.25) * 100);
    }

    /**
     * Density: what percentage of the canvas is covered by elements.
     * Ideal range 20–60%, outside that loses points.
     */
    private static int scoreDensity(List<DesignElement> elements, double cW, double cH) {
        if (elements.isEmpty()) return 0;
        double totalArea  = elements.stream()
                .mapToDouble(el -> el.getWidth() * el.getHeight())
                .sum();
        double coverage = (totalArea / (cW * cH)) * 100;
        // Ideal: 20–60%
        if (coverage >= 20 && coverage <= 60) return 100;
        if (coverage < 20) return (int)(coverage / 20.0 * 100);
        // > 60%: penalise for crowding
        return (int) Math.max(0, 100 - (coverage - 60) * 2);
    }

    /**
     * Complexity: element count.
     * 3–12 elements = ideal, fewer or more loses points.
     */
    private static int scoreComplexity(List<DesignElement> elements) {
        int n = elements.size();
        if (n == 0) return 10;
        if (n >= 3 && n <= 12) return 100;
        if (n < 3)  return (int)(n / 3.0 * 100);
        return (int) Math.max(20, 100 - (n - 12) * 5);
    }

    // ── Feedback text ──────────────────────────────────────────────

    private static String feedbackBalance(int score) {
        if (score >= 80) return "Well balanced layout";
        if (score >= 50) return "Slightly off-center";
        return "Layout is visually heavy on one side";
    }

    private static String feedbackContrast(int score) {
        if (score >= 75) return "Good color variety";
        if (score >= 40) return "Moderate contrast";
        return "Colors are too similar — add contrast";
    }
```

```
    private static String feedbackDensity(int score) {
        if (score >= 80) return "Canvas coverage is ideal";
        if (score >= 40) return "Could use more or fewer elements";
        return score < 40 ? "Canvas feels sparse" : "Canvas feels crowded";
    }

    private static String feedbackComplexity(int score, int count) {
        if (score >= 80) return "Good element count (" + count + ")";
        if (count < 3)  return "Too few elements — add more";
        return "Too many elements — simplify";
    }

    private static String overallTag(int score) {
        if (score >= 85) return "Excellent — publication ready!";
        if (score >= 65) return "Good — minor tweaks suggested";
        if (score >= 45) return "Fair — room for improvement";
        return "Needs work — review feedback below";
    }

    // ── UI helpers ──────────────────────────────────────────────────────

    private static HBox scoreRow(String label, int score, String feedback) {
        Label nameLbl = new Label(label);
        nameLbl.setStyle("-fx-text-fill: #e2a9f1; -fx-font-family: 'Segoe UI'; " +
                "-fx-font-weight: bold; -fx-font-size: 12px;");
        nameLbl.setPrefWidth(100);

        // Bar
        StackPane barBg = new StackPane();
        barBg.setPrefWidth(100);
        barBg.setPrefHeight(8);
        barBg.setStyle("-fx-background-color: #2b2b4a; -fx-background-radius: 4;");

        Rectangle fill = new Rectangle(score, 8);
        fill.setFill(Color.web(scoreColor(score)));
        fill.setArcWidth(4);
        fill.setArcHeight(4);
        StackPane.setAlignment(fill, Pos.CENTER_LEFT);
        barBg.getChildren().add(fill);

        Label scoreLbl = new Label(score + "%");
        scoreLbl.setStyle("-fx-text-fill: " + scoreColor(score) + "; " +
                "-fx-font-family: 'Segoe UI'; -fx-font-weight: bold; -fx-font-size: 11px;");
        scoreLbl.setPrefWidth(38);
        scoreLbl.setAlignment(Pos.CENTER_RIGHT);

        Label feedbackLbl = new Label(feedback);
        feedbackLbl.setStyle("-fx-text-fill: #888; -fx-font-family: 'Segoe UI'; -fx-font-size: 10px;");

        VBox left = new VBox(2, new HBox(6, nameLbl, barBg, scoreLbl), feedbackLbl);
        HBox row = new HBox(left);
        return row;
    }

    private static String scoreColor(int score) {
        if (score >= 75) return "#43e8a0";
        if (score >= 50) return "#ffd166";
        return "#fb6c6a";
    }

    private static double luminance(String hex) {
        try {
            Color c = Color.web(hex);
            return 0.2126 * c.getRed() + 0.7152 * c.getGreen() + 0.0722 * c.getBlue();
        } catch (Exception e) {
            return 0.5;
        }
    }
}
```

TemplateEngine.java

Programmatically constructs five design templates (Business Card, Poster, Social Post, Presentation Slide, Logo Draft) as lists of DesignElement objects without any external image assets.

```
// TemplateEngine.java
package com.framestudyo.framestudyo;

import java.util.ArrayList;
import java.util.List;


public class TemplateEngine {

    // Canvas dimensions assumed by the templates
    private static final double CW = 860;
    private static final double CH = 600;

    // ── Template catalogue ──────────────────────────────────────

    public enum Template {
        BUSINESS_CARD  ("💼 Business Card",  "Name, title, contact layout"),
        POSTER         ("🖼 Poster",         "Headline, sub-text, accent shapes"),
        SOCIAL_POST    ("📱 Social Post",    "Square composition with header"),
        PRESENTATION   ("📊 Slide",          "Title slide with content block"),
        LOGO_DRAFT     ("✦ Logo Draft",      "Centered symbol + wordmark area"),
        BLANK          ("⬡ Blank Canvas",   "Start from scratch");

        private final String displayName;
        private final String description;

        Template(String displayName, String description) {
            this.displayName = displayName;
            this.description = description;
        }

        public String displayName() { return displayName; }
        public String description() { return description; }
    }

    public static List<Template> getTemplates() {
        return List.of(Template.values());
    }

    // ── Builder dispatch ────────────────────────────────────────

    public static List<DesignElement> build(Template t) {
        return switch (t) {
            case BUSINESS_CARD -> buildBusinessCard();
            case POSTER        -> buildPoster();
            case SOCIAL_POST   -> buildSocialPost();
            case PRESENTATION  -> buildPresentation();
            case LOGO_DRAFT    -> buildLogoDraft();
            case BLANK         -> new ArrayList<>();
        };
    }

    // ── Templates ───────────────────────────────────────────────

    private static List<DesignElement> buildBusinessCard() {
        List<DesignElement> els = new ArrayList<>();

        // Background card
        RectangleElement bg = new RectangleElement(80, 150, 700, 300);
        bg.setFillColor("#1a1a2e");
```

```
        bg.setStrokeColor("#3ddcf8");
        bg.setStrokeWidth(2);
        els.add(bg);

        // Accent bar left
        RectangleElement bar = new RectangleElement(80, 150, 8, 300);
        bar.setFillColor("#ac7cc3");
        bar.setStrokeColor("transparent");
        bar.setStrokeWidth(0);
        els.add(bar);

        // Name text
        TextElement name = new TextElement(140, 220, "Your Name");
        name.setFillColor("#ffffff");
        name.setFontFamily("Segoe UI");
        name.setFontSize(32);
        els.add(name);

        // Title
        TextElement jobTitle = new TextElement(142, 270, "Job Title  |  Company");
        jobTitle.setFillColor("#ac7cc3");
        name.setFontFamily("Segoe UI");
        name.setFontSize(16);
        els.add(jobTitle);

        // Divider line
        LineElement divider = new LineElement(140, 300, 740, 300);
        divider.setStrokeColor("#3ddcf8");
        divider.setStrokeWidth(1);
        els.add(divider);

        // Contact info
        TextElement email = new TextElement(142, 330, "email@example.com");
        email.setFillColor("#cccccc");
        name.setFontFamily("Segoe UI");
        name.setFontSize(13);
        els.add(email);

        TextElement phone = new TextElement(142, 360, "+92 300 0000000");
        phone.setFillColor("#cccccc");
        name.setFontFamily("Segoe UI");
        name.setFontSize(13);
        els.add(phone);

        // Decorative circle (logo placeholder)
        CircleElement logo = new CircleElement(660, 185, 100, 100);
        logo.setFillColor("#ac7cc3");
        logo.setStrokeColor("#3ddcf8");
        logo.setStrokeWidth(2);
        els.add(logo);

        TextElement logoTxt = new TextElement(690, 228, "LOGO");
        logoTxt.setFillColor("#ffffff");
        name.setFontFamily("Segoe UI");
        name.setFontSize(14);
        els.add(logoTxt);

        return els;
    }

    private static List<DesignElement> buildPoster() {
        List<DesignElement> els = new ArrayList<>();

        // Background
        RectangleElement bg = new RectangleElement(40, 20, 780, 560);
        bg.setFillColor("#0f0e17");
        bg.setStrokeColor("#eb6891");
        bg.setStrokeWidth(3);
        els.add(bg);
```

```
        // Top accent strip
        RectangleElement strip = new RectangleElement(40, 20, 780, 8);
        strip.setFillColor("#eb6891");
        strip.setStrokeColor("transparent");
        strip.setStrokeWidth(0);
        els.add(strip);

        // Headline
        TextElement headline = new TextElement(120, 120, "YOUR HEADLINE");
        headline.setFillColor("#ffffff");
        headline.setFontFamily("Segoe UI");
        headline.setFontSize(32);
        els.add(headline);

        // Subheadline
        TextElement sub = new TextElement(155, 185, "Subtitle — Date — Location");
        sub.setFillColor("#eb6891");
        headline.setFontFamily("Segoe UI");
        headline.setFontSize(20);
        els.add(sub);

        // Divider line
        LineElement div = new LineElement(100, 210, 760, 210);
        div.setStrokeColor("#eb6891");
        div.setStrokeWidth(1.5);
        els.add(div);

        // Body text placeholder
        TextElement body = new TextElement(115, 260, "Description or body text goes here.");
        body.setFillColor("#aaaaaa");
        body.setFontFamily("Segoe UI");
        body.setFontSize(16);
        els.add(body);

        // Decorative star accent
        StarElement star1 = new StarElement(680, 310, 80, 80);
        star1.setFillColor("#eb6891");
        star1.setStrokeColor("transparent");
        star1.setStrokeWidth(0);
        star1.setOpacity(0.4);
        els.add(star1);

        StarElement star2 = new StarElement(100, 380, 50, 50);
        star2.setFillColor("#3ddcf8");
        star2.setStrokeColor("transparent");
        star2.setStrokeWidth(0);
        star2.setOpacity(0.3);
        els.add(star2);

        // CTA box
        RectangleElement cta = new RectangleElement(280, 460, 300, 60);
        cta.setFillColor("#eb6891");
        cta.setStrokeColor("transparent");
        cta.setStrokeWidth(0);
        els.add(cta);

        TextElement ctaTxt = new TextElement(345, 482, "REGISTER NOW");
        ctaTxt.setFillColor("#ffffff");
        ctaTxt.setFontFamily("Segoe UI");
        ctaTxt.setFontSize(18);
        els.add(ctaTxt);

        return els;
    }

    private static List<DesignElement> buildSocialPost() {
        List<DesignElement> els = new ArrayList<>();

        // Square background (Instagram-ish)
        RectangleElement bg = new RectangleElement(180, 30, 500, 500);
        bg.setFillColor("#16213e");
```

```
        bg.setStrokeColor("#ac7cc3");
        bg.setStrokeWidth(2);
        els.add(bg);

        // Header bar
        RectangleElement header = new RectangleElement(180, 30, 500, 70);
        header.setFillColor("#ac7cc3");
        header.setStrokeColor("transparent");
        header.setStrokeWidth(0);
        els.add(header);

        TextElement handle = new TextElement(195, 55, "@yourhandle");
        handle.setFillColor("#ffffff");
        handle.setFontFamily("Segoe UI");
        handle.setFontSize(18);
        els.add(handle);

        // Main image placeholder
        RectangleElement imgPlaceholder = new RectangleElement(200, 120, 460, 260);
        imgPlaceholder.setFillColor("#0f3460");
        imgPlaceholder.setStrokeColor("#3ddcf8");
        imgPlaceholder.setStrokeWidth(1);
        els.add(imgPlaceholder);

        TextElement imgTxt = new TextElement(340, 255, "Image");
        imgTxt.setFillColor("#3ddcf8");
        imgTxt.setFontFamily("Segoe UI");
        imgTxt.setFontSize(20);
        els.add(imgTxt);

        // Caption
        TextElement caption = new TextElement(200, 420, "Your caption goes here ✨");
        caption.setFillColor("#ffffff");
        caption.setFontFamily("Segoe UI");
        caption.setFontSize(15);
        els.add(caption);

        // Hashtag line
        TextElement tags = new TextElement(200, 455, "#design  #creative  #framestudyo");
        tags.setFillColor("#ac7cc3");
        tags.setFontFamily("Segoe UI");
        tags.setFontSize(12);
        els.add(tags);

        return els;
    }

    private static List<DesignElement> buildPresentation() {
        List<DesignElement> els = new ArrayList<>();

        // Slide background
        RectangleElement bg = new RectangleElement(30, 30, 800, 540);
        bg.setFillColor("#ffffff");
        bg.setStrokeColor("#cccccc");
        bg.setStrokeWidth(1);
        els.add(bg);

        // Left accent band
        RectangleElement band = new RectangleElement(30, 30, 12, 540);
        band.setFillColor("#3e7e9f");
        band.setStrokeColor("transparent");
        band.setStrokeWidth(0);
        els.add(band);

        // Title
        TextElement slideTitle = new TextElement(70, 110, "Presentation Title");
        slideTitle.setFillColor("#1a1a2e");
        slideTitle.setFontFamily("Segoe UI");
        slideTitle.setFontSize(36);
        els.add(slideTitle);
```

```
        // Underline
        LineElement underline = new LineElement(70, 155, 500, 155);
        underline.setStrokeColor("#3e7e9f");
        underline.setStrokeWidth(3);
        els.add(underline);

        // Subtitle
        TextElement subtitle = new TextElement(70, 185, "Subtitle or presenter name");
        subtitle.setFillColor("#666666");
        subtitle.setFontFamily("Segoe UI");
        subtitle.setFontSize(18);
        els.add(subtitle);

        // Content area placeholder
        RectangleElement content = new RectangleElement(70, 230, 460, 280);
        content.setFillColor("#f5f5f5");
        content.setStrokeColor("#cccccc");
        content.setStrokeWidth(1);
        els.add(content);

        TextElement contentTxt = new TextElement(85, 280, "• Bullet point one");
        contentTxt.setFillColor("#333333");
        contentTxt.setFontFamily("Segoe UI");
        contentTxt.setFontSize(15);
        els.add(contentTxt);

        TextElement contentTxt2 = new TextElement(85, 320, "• Bullet point two");
        contentTxt2.setFillColor("#333333");
        contentTxt2.setFontFamily("Segoe UI");
        contentTxt2.setFontSize(15);
        els.add(contentTxt2);

        TextElement contentTxt3 = new TextElement(85, 360, "• Bullet point three");
        contentTxt3.setFillColor("#333333");
        contentTxt3.setFontFamily("Segoe UI");
        contentTxt3.setFontSize(15);
        els.add(contentTxt3);

        // Image placeholder (right side)
        RectangleElement imgBox = new RectangleElement(560, 230, 240, 280);
        imgBox.setFillColor("#ddeeff");
        imgBox.setStrokeColor("#3e7e9f");
        imgBox.setStrokeWidth(1);
        els.add(imgBox);

        TextElement imgLbl = new TextElement(625, 372, "Image");
        imgLbl.setFillColor("#3e7e9f");
        imgLbl.setFontFamily("Segoe UI");
        imgLbl.setFontSize(16);
        els.add(imgLbl);

        // Footer
        RectangleElement footer = new RectangleElement(30, 545, 800, 25);
        footer.setFillColor("#3e7e9f");
        footer.setStrokeColor("transparent");
        footer.setStrokeWidth(0);
        els.add(footer);

        TextElement footerTxt = new TextElement(40, 552, "FrameStudyo  |  Page 1");
        footerTxt.setFillColor("#ffffff");
        footerTxt.setFontFamily("Segoe UI");
        footerTxt.setFontSize(11);
        els.add(footerTxt);

        return els;
    }

    private static List<DesignElement> buildLogoDraft() {
        List<DesignElement> els = new ArrayList<>();
```

```
        // Outer circle (ring)
        CircleElement outerRing = new CircleElement(305, 130, 250, 250);
        outerRing.setFillColor("transparent");
        outerRing.setStrokeColor("#ac7cc3");
        outerRing.setStrokeWidth(4);
        els.add(outerRing);

        // Inner filled circle
        CircleElement inner = new CircleElement(355, 180, 150, 150);
        inner.setFillColor("#ac7cc3");
        inner.setStrokeColor("transparent");
        inner.setStrokeWidth(0);
        els.add(inner);

        // Center star
        StarElement star = new StarElement(385, 210, 90, 90);
        star.setFillColor("#ffffff");
        star.setStrokeColor("transparent");
        star.setStrokeWidth(0);
        els.add(star);

        // Brand name below
        TextElement brand = new TextElement(290, 420, "BRAND NAME");
        brand.setFillColor("#1a1a2e");
        brand.setFontFamily("Segoe UI");
        brand.setFontSize(40);
        els.add(brand);

        // Tagline
        TextElement tagline = new TextElement(330, 468, "Your tagline here");
        tagline.setFillColor("#ac7cc3");
        tagline.setFontFamily("Segoe UI");
        tagline.setFontSize(15);
        els.add(tagline);

        // Decorative lines either side
        LineElement lineL = new LineElement(100, 445, 280, 445);
        lineL.setStrokeColor("#ac7cc3");
        lineL.setStrokeWidth(1.5);
        els.add(lineL);

        LineElement lineR = new LineElement(580, 445, 760, 445);
        lineR.setStrokeColor("#ac7cc3");
        lineR.setStrokeWidth(1.5);
        els.add(lineR);

        return els;
    }
}
```

SessionTracker.java

Singleton session analytics. Records session start time, maintains an ordered action log, and computes live statistics including shape-type counts, most-used colour, and canvas coverage.

```
// SessionTracker.java
package com.framestudyo.framestudyo;

import java.time.LocalDateTime;
import java.time.Duration;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.stream.Collectors;



;
```

```
public class SessionTracker {

    // ── Singleton ──────────────────────────────────────────────────────

    private static SessionTracker instance;

    public static SessionTracker getInstance() {
        if (instance == null) instance = new SessionTracker();
        return instance;
    }

    private SessionTracker() {}

    // ── State ──────────────────────────────────────────────────────────

    private final List<ActionEntry> history = new ArrayList<>();
    private LocalDateTime sessionStart;
    private long accumulatedPausedMs = 0; // total ms spent paused
    private long pauseStartMs = -1;       // wall-clock ms when current pause began
  //  private boolean running = false;

    private static final DateTimeFormatter FMT =
            DateTimeFormatter.ofPattern("HH:mm:ss");
    private static final DateTimeFormatter DATE_FMT =
            DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");

    // ── Session control ────────────────────────────────────────────────

    public void startSession() {
        sessionStart        = LocalDateTime.now();
        accumulatedPausedMs = 0;
        pauseStartMs        = -1;
        //running             = true;
        history.clear();
        logAction("Session started");
    }

   /* public void pauseSession() {
       // if (running) {
            pauseStartMs = System.currentTimeMillis();
            running = false;
        }
    }*/

   /* public void resumeSession() {
        if (!running && pauseStartMs >= 0) {
            accumulatedPausedMs += System.currentTimeMillis() - pauseStartMs;
            pauseStartMs = -1;
            running = true;
            logAction("Returned to canvas");
        }
    }
*/
    /**
     * Returns total ACTIVE milliseconds (wall time minus all paused time).
     * Works correctly whether the session is currently running or paused.
     */
    public long elapsedMs() {

        if (sessionStart == null)
            return 0;

        return Duration.between(
                sessionStart,
                LocalDateTime.now()
        ).toMillis();
    }

    // ── Action logging ─────────────────────────────────────────────────
```

```
    public void logAction(String description) {
        history.add(new ActionEntry(LocalDateTime.now(), description));
    }

    /** Returns the full action log (oldest first). */
    public List<ActionEntry> getHistory() {
        return Collections.unmodifiableList(history);
    }

    /** Returns the last n actions (oldest first). */
    public List<ActionEntry> getRecentHistory(int n) {
        int from = Math.max(0, history.size() - n);
        return Collections.unmodifiableList(history.subList(from, history.size()));
    }

    // ── Statistics ──────────────────────────────────────────────

    public SessionStats computeStats(DesignCanvas designCanvas) {
        List<DesignElement> elements = designCanvas.getElements();

        // Shape type counts
        Map<String, Long> shapeCounts = elements.stream()
                .collect(Collectors.groupingBy(
                        el -> el.getClass().getSimpleName().replace("Element", ""),
                        Collectors.counting()
                ));

        // Color frequency
        Map<String, Long> colorCounts = elements.stream()
                .filter(el -> el.getFillColor() != null
                        && !el.getFillColor().isEmpty()
                        && !el.getFillColor().equalsIgnoreCase("transparent"))
                .collect(Collectors.groupingBy(
                        el -> el.getFillColor().toUpperCase(),
                        Collectors.counting()
                ));

        String mostUsedColor = colorCounts.entrySet().stream()
                .max(Map.Entry.comparingByValue())
                .map(Map.Entry::getKey)
                .orElse("None");

        // Canvas coverage
        double totalArea = elements.stream()
                .mapToDouble(el -> el.getWidth() * el.getHeight())
                .sum();
        double coveragePct = Math.min(100.0, (totalArea / (860.0 * 600.0)) * 100.0);

        // Format elapsed time
        long elapsed = elapsedMs();
        long minutes = elapsed / 60000;
        long seconds = (elapsed % 60000) / 1000;
        String timeStr = minutes + "m " + seconds + "s";

        String startStr = (sessionStart != null)
                ? sessionStart.format(DATE_FMT) : "Not started";

        return new SessionStats(
                elements.size(),
                shapeCounts,
                colorCounts,
                mostUsedColor,
                coveragePct,
                history.size(),
                timeStr,
                startStr
        );
    }
```

```
    // ── Inner classes ────────────────────────────────────────────────

    public static class ActionEntry {
        private final LocalDateTime timestamp;
        private final String description;

        public ActionEntry(LocalDateTime timestamp, String description) {
            this.timestamp   = timestamp;
            this.description = description;
        }

        public String getDescription()      { return description; }
        public LocalDateTime getTimestamp() { return timestamp; }

        @Override
        public String toString() {
            return "[" + timestamp.format(FMT) + "]  " + description;
        }
    }

    public static class SessionStats {
        public final int totalElements;
        public final Map<String, Long> shapeCounts;
        public final Map<String, Long> colorCounts;
        public final String mostUsedColor;
        public final double coveragePercent;
        public final int totalActions;
        public final String sessionTime;
        public final String sessionStart;

        public SessionStats(int totalElements,
                            Map<String, Long> shapeCounts,
                            Map<String, Long> colorCounts,
                            String mostUsedColor,
                            double coveragePercent,
                            int totalActions,
                            String sessionTime,
                            String sessionStart) {
            this.totalElements   = totalElements;
            this.shapeCounts     = shapeCounts;
            this.colorCounts     = colorCounts;
            this.mostUsedColor   = mostUsedColor;
            this.coveragePercent = coveragePercent;
            this.totalActions    = totalActions;
            this.sessionTime     = sessionTime;
            this.sessionStart    = sessionStart;
        }
    }
}
```

FileManager.java

Handles ObjectOutputStream serialisation to .fsd files and ObjectInputStream deserialisation with robust error handling for version mismatches and corrupt files.

```
// FileManager.java
package com.framestudyo.framestudyo;

import java.io.*;
import java.util.List;

public class FileManager {

    public static void save(List<DesignElement> elements, String filePath) {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(filePath))) {
            out.writeObject(elements);
            System.out.println("Saved successfully!");
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
```

```
        }
    }

    public static List<DesignElement> load(String filePath) {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(filePath))) {
            List<DesignElement> elements = (List<DesignElement>) in.readObject();
            System.out.println("Loaded successfully! Elements: " + elements.size());
            return elements;
        } catch (InvalidClassException e) {
            System.out.println("File format outdated — please save a new file: " + e.getMessage());
            showError("This file was saved with an older version of FrameStudyo and can't be loaded.
Please start a new design.");
            return null;
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error loading: " + e.getMessage());
            showError("Could not load file: " + e.getMessage());
            return null;
        }
    }

    private static void showError(String message) {
        javafx.application.Platform.runLater(() -> {
            javafx.scene.control.Alert alert = new javafx.scene.control.Alert(
                    javafx.scene.control.Alert.AlertType.ERROR);
            alert.setTitle("Load Error");
            alert.setHeaderText("Could not load design");
            alert.setContentText(message);
            alert.showAndWait();
        });
    }
}
```

9. Testing & Known Limitations

9.1 Testing Approach

Testing was conducted manually through exploratory session testing during development. Each feature was verified against the following scenarios:

- Shape drawing – All six shape types drawn at various sizes and positions.
- Freehand recognition – Circles, rectangles, triangles, lines, and stars drawn freehand and verified for correct classification.
- Undo/Redo – Sequential undo down to empty canvas and redo back to last state.
- Save/Load – Design saved as .fsd and reloaded; all element positions, colours, and properties verified.
- PNG Export – Exported image compared pixel-for-pixel against the on-screen canvas.
- Templates – All five templates loaded and confirmed to render correctly.
- Smart tools – Colour Advisor, Design Analyzer, and Alignment Engine verified for correctness.

9.2 Known Limitations

- File compatibility: The .fsd format is tied to the Java class serialVersionUID. Adding new fields to any DesignElement subclass will invalidate existing saved files.
- ImageElement path dependency: Images are stored by absolute file path. Moving or renaming the source image file after saving will cause the image to fail to load on reload.
- Text bounding box: TextElement uses a fixed 200x50 bounding box for hit-testing, which may not match the rendered text dimensions exactly for very large or very small font sizes.
- Rotation and resize: Corner-handle resizing does not account for element rotation; resizing a rotated element will resize the axis-aligned bounding box rather than the rotated frame.

10. Conclusion

FrameStudyo successfully delivers a fully functional, visually polished desktop graphic design application built entirely in Java and JavaFX. The project demonstrates mastery of object-oriented programming through a deep class hierarchy, the Command design pattern for safe undo/redo, singleton-based analytics, and a suite of intelligent tools that go well beyond basic shape drawing.

The application was designed, coded, and demonstrated as an academic project. Every class in the codebase serves a clearly defined single responsibility, making the system straightforward to extend — adding a new shape requires only creating a new DesignElement subclass and registering it in the Shapes dropdown.

Future work could include multi-page canvas support, PDF export, cloud save integration, font preview in the text dialog, and a proper SVG import/export pipeline.

End of Report